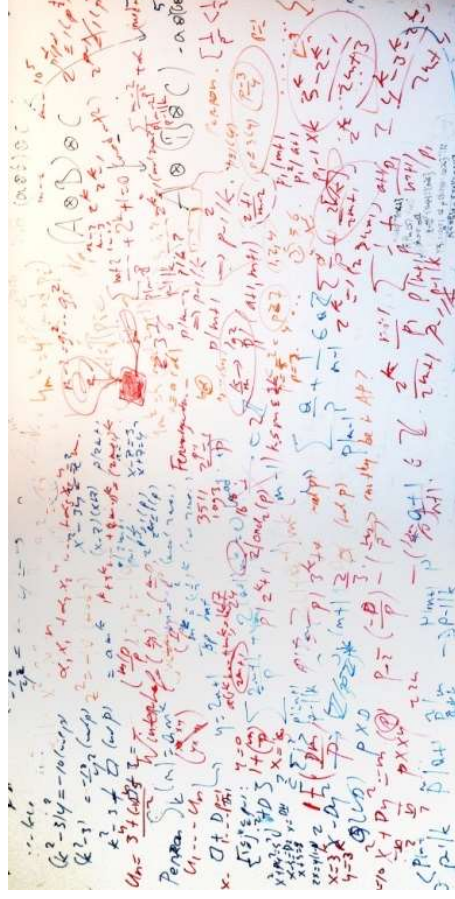


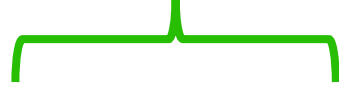
1) 2)



Building A Model

5 methods of building models:

1. All-in
2. Backward Elimination
3. Forward Selection
4. Bidirectional Elimination
5. Score Comparison



Stepwise
Regression

Building A Model

“All-in” – cases:

- Prior knowledge; OR
- You have to; OR
- Preparing for Backward Elimination



Building A Model

Backward Elimination

STEP 1: Select a significance level to stay in the model (e.g. $SL = 0.05$)



STEP 2: Fit the full model with all possible predictors



STEP 3: Consider the predictor with the highest P-value. If $P > SL$, go to STEP 4, otherwise



STEP 4: Remove the predictor



STEP 5: Fit model without this variable*



FIN: Your Model Is

Building A Model

Forward Selection

STEP 1: Select a significance level to enter the model (e.g. $SL = 0.05$)



STEP 2: Fit all simple regression models $y \sim x_n$ Select the one with the lowest P-value



STEP 3: Keep this variable and fit all possible models with one extra predictor added to already have



STEP 4: Consider the predictor with the lowest P-value. If $P < SL$, go to STEP 3, otherwise



FIN: Keep the previous

Building A Model

Bidirectional Elimination

STEP 1: Select a significance level to enter and to stay in the model
e.g.: SLENTER = 0.05, SLSTAY = 0.05



STEP 2: Perform the next step of Forward Selection (new variables must have: $P < SLENTER$)



STEP 3: Perform ALL steps of Backward Elimination (old variables must have $P < SLSTAY$)



STEP 4: No new variables can enter and no old variables can exit



FIN: Your Model Is Ready

Building A Model

All Possible Models

STEP 1: Select a criterion of goodness of fit (e.g. Akaike criterion)



STEP 2: Construct All Possible Regression Models: 2^{N-1} total combinations



STEP 3: Select the one with the best criterion



FIN: Your Model Is Ready



Example:
10 columns means
1,023 models

Building A Model

5 methods of building models:

1. All-in
2. Backward Elimination
3. Forward Selection
4. Bidirectional Elimination
5. Score Comparison

Section Recap

Section Recap

In this section we learned:

1. How to create dummies for categorical IVs
2. How to avoid the dummy variable trap
3. Backward, Forward, Bidirectional, All Possible
4. We actually built a model. Step-By-Step!!
5. How to use adjusted R-squared in modelling
6. How to interpret coefficients of a MLR

Polynomial Regression

Regressions

Simple
Linear
Regression

$$y = b_0 + b_1 x_1$$

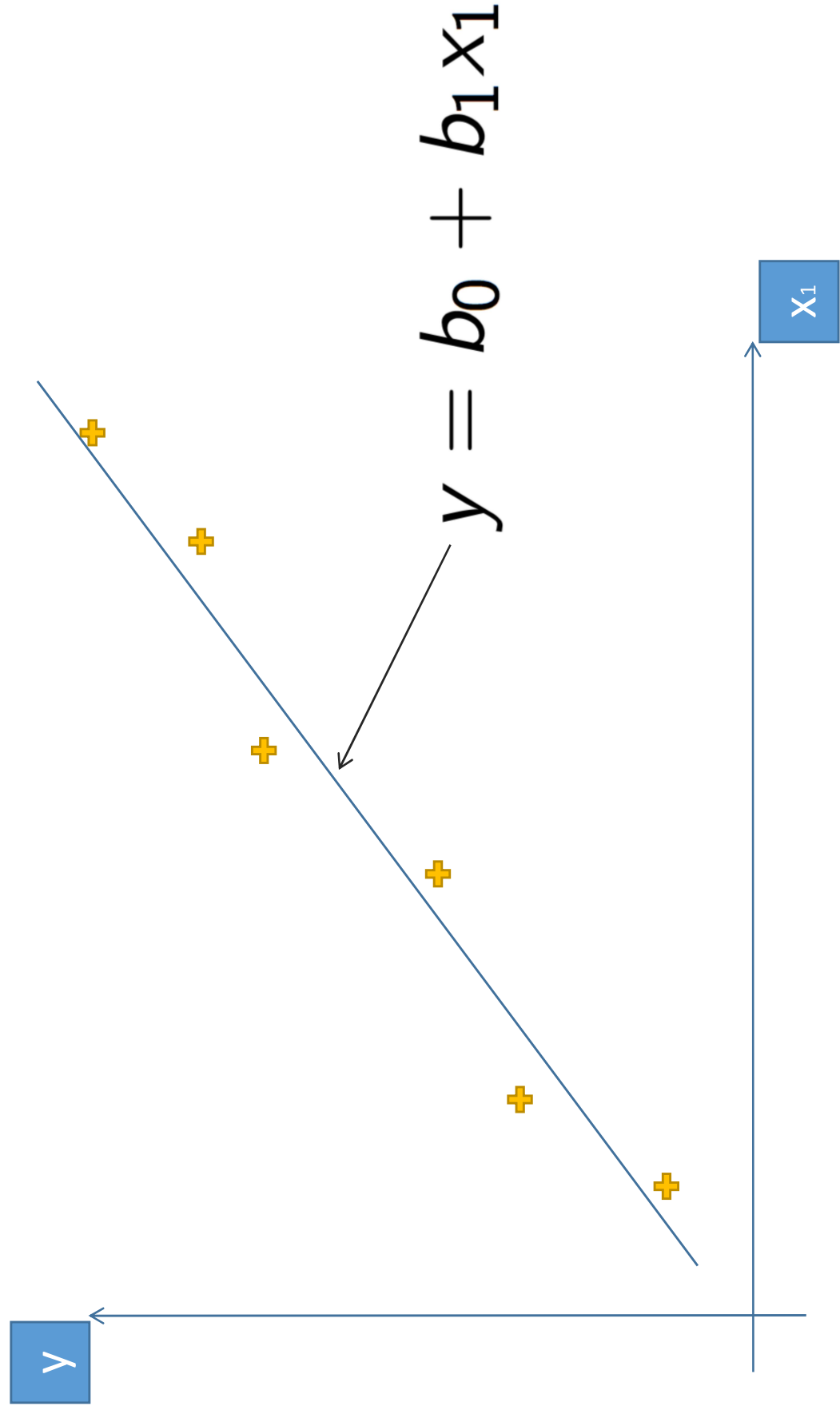
Multiple
Linear
Regression

$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots$$

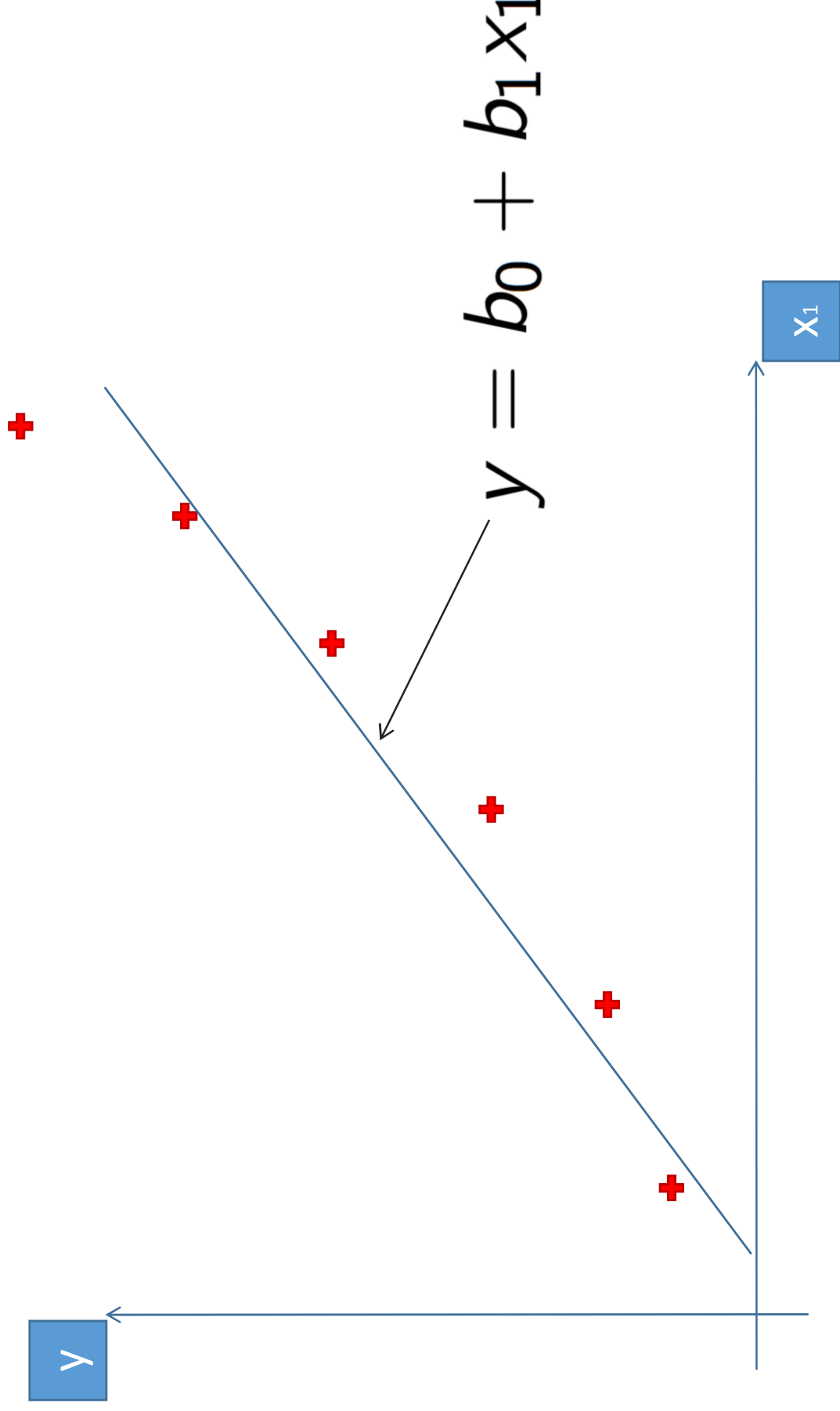
Polynomial
Linear
Regression

$$y = b_0 + b_1 x_1 + b_2 x_1^2 + \dots$$

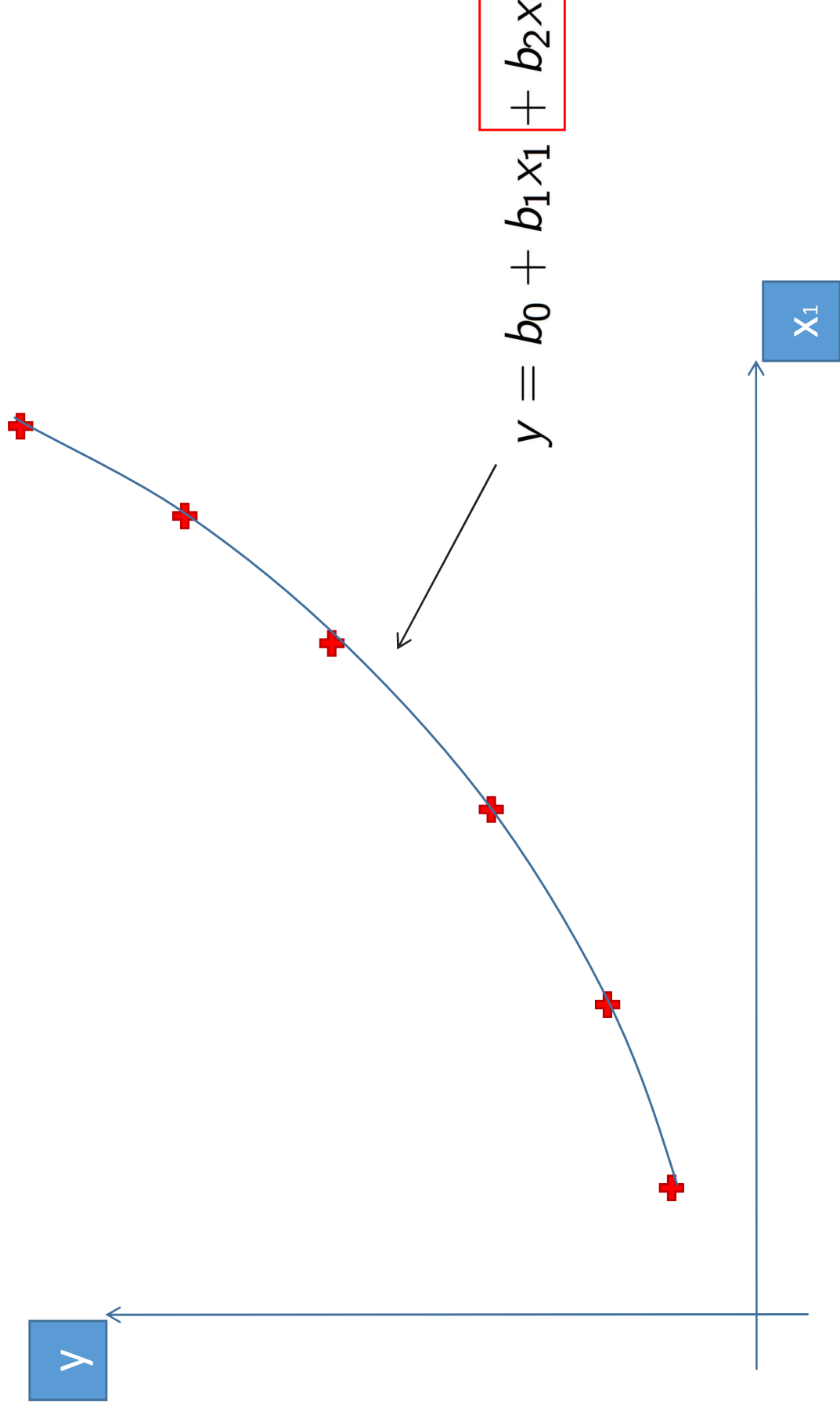
Simple Linear Regression



Simple Linear Regression



Polynomial Regression



Polynomial Regression

One Question: Why “Linear”?

Polynomial Regression

Polynomial
Linear
Regression

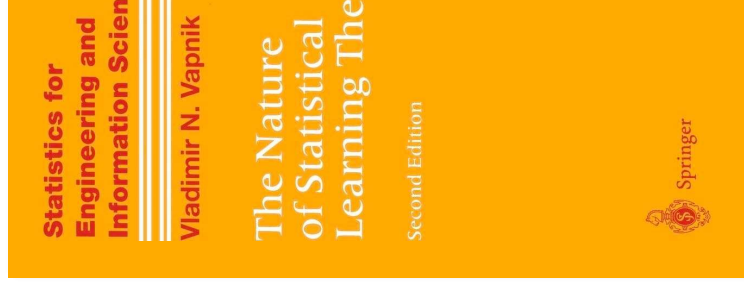
$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots$$

SVR Intuition

SVR Intuition



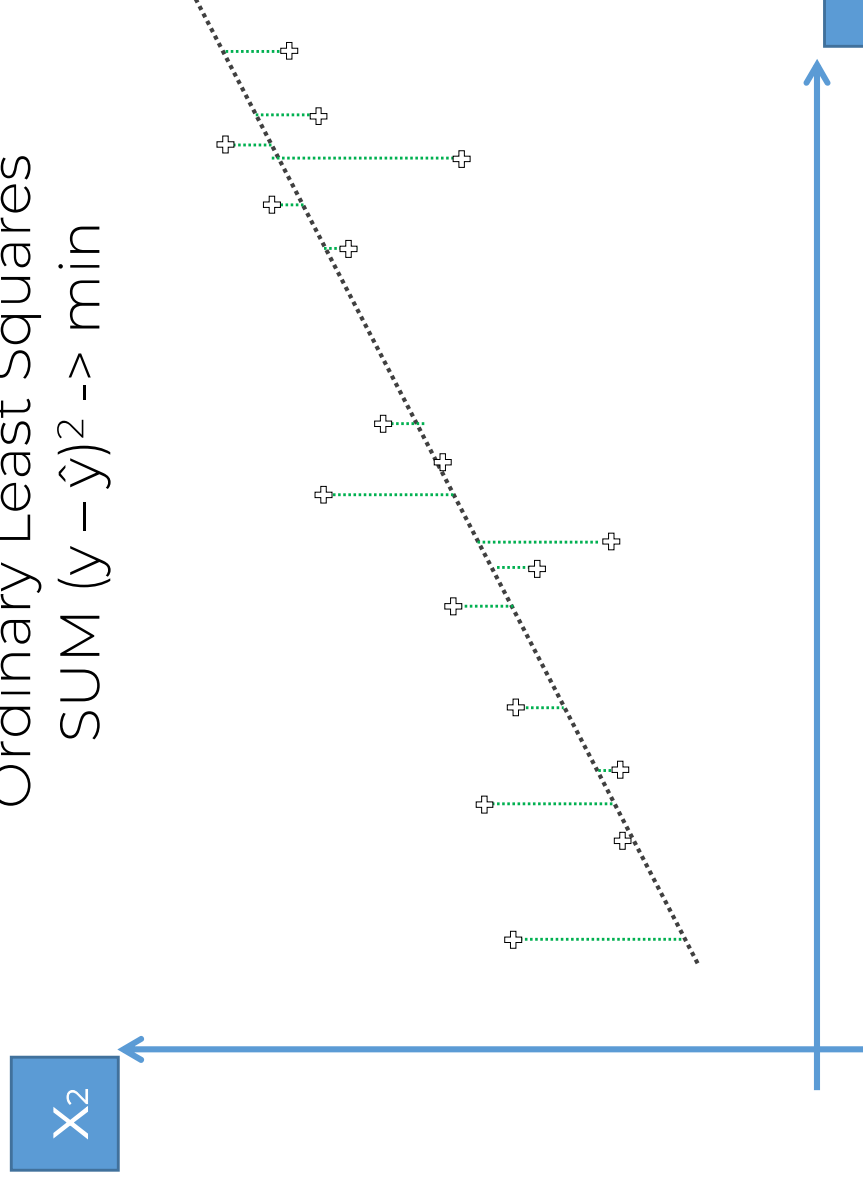
Vladimir Vapnik



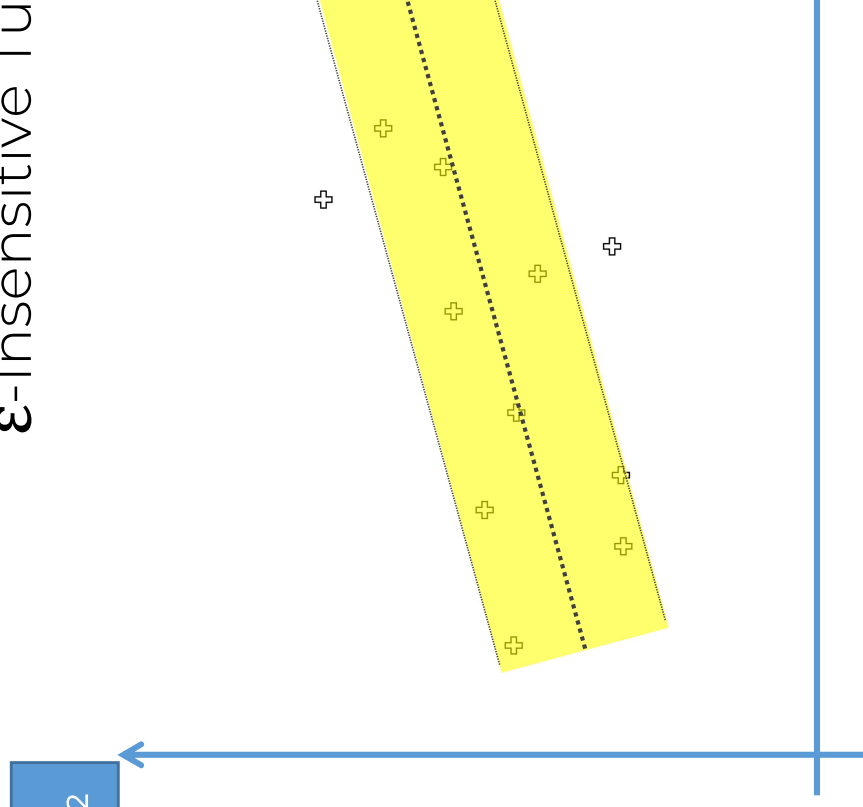
199

SVR Intuition

Ordinary Least Squares
 $\text{SUM } (y - \hat{y})^2 \rightarrow \min$



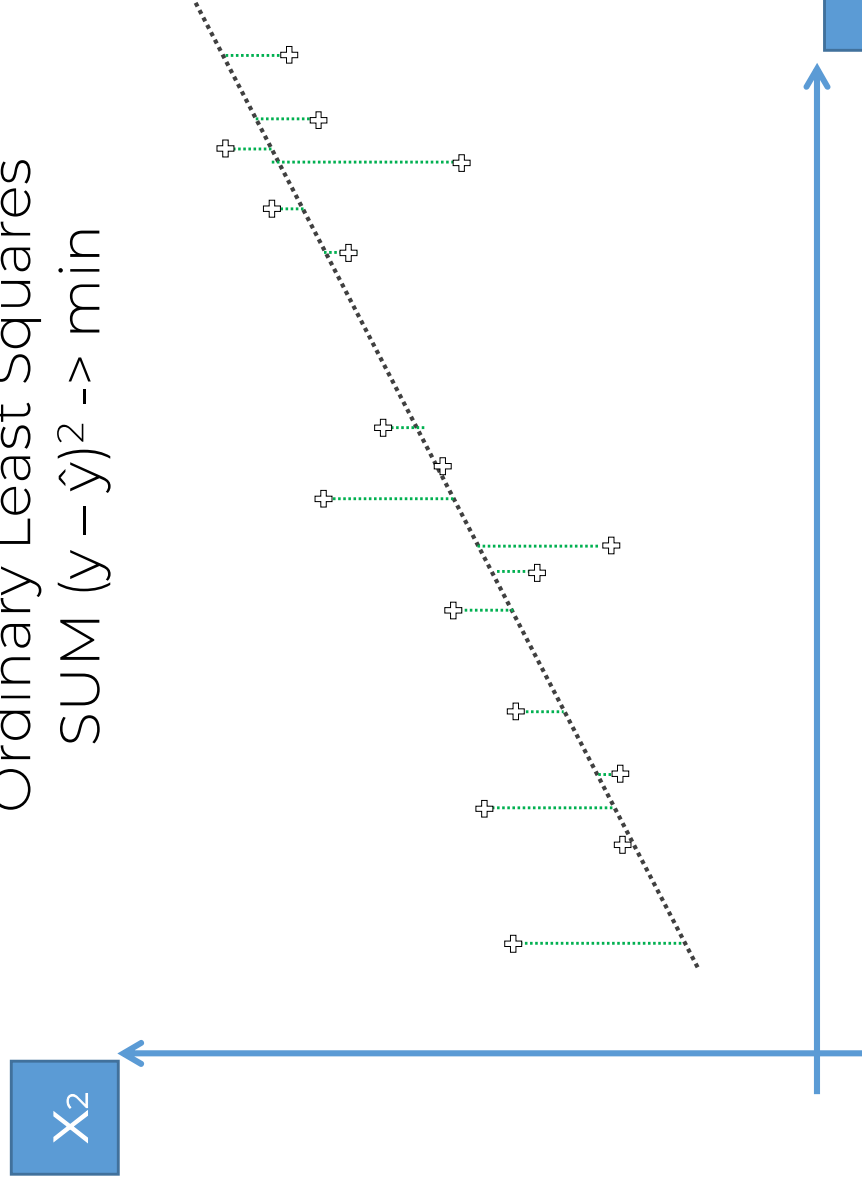
ϵ -Insensitive Tu



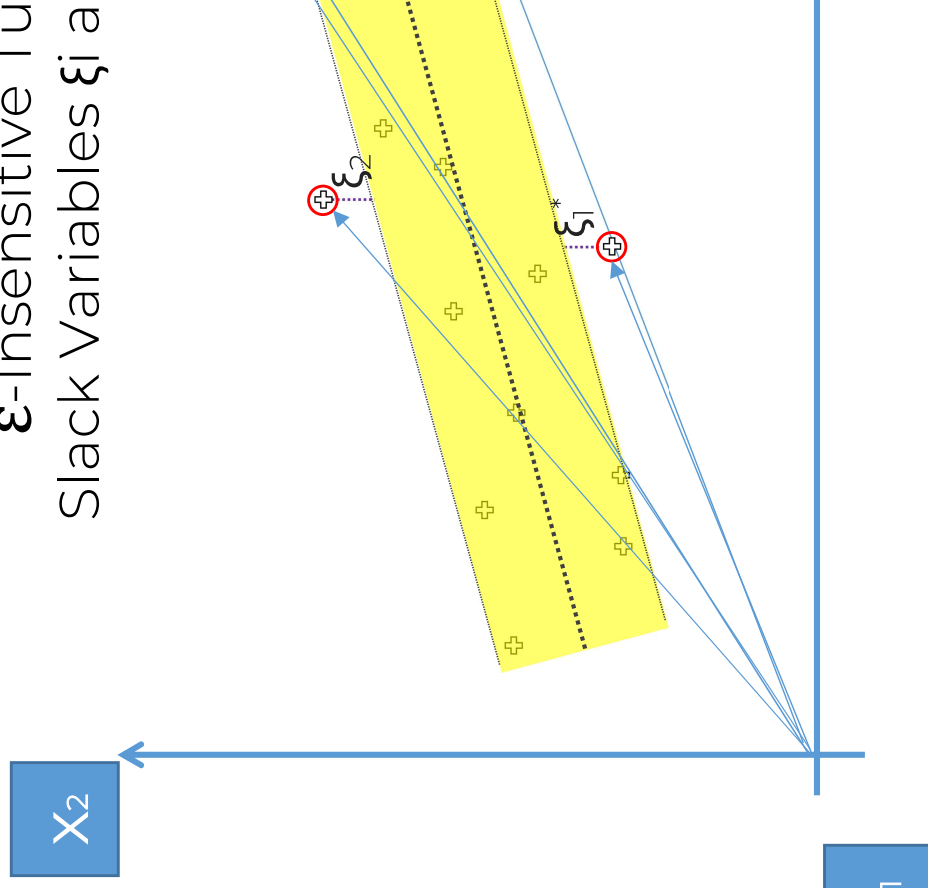
SVR Intuition

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^m (\xi_i + \xi_i^*)$$

Ordinary Least Squares
 $\text{SUM } (y - \hat{y})^2 \rightarrow \min$



ϵ -Insensitive Tu
 Slack Variables ξ_i a



SVR Intuition

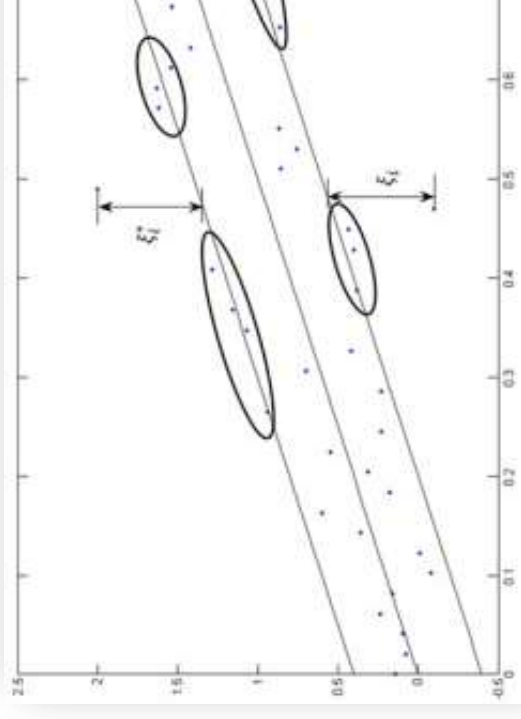
Additional Reading:

Chapter 4 – Support Vector Regression
(from: *Efficient Learning Machines: Theories, Concepts, and Applications for Engineers and System Designers*)

By Mariette Awad & Rahul Khanna (2015)

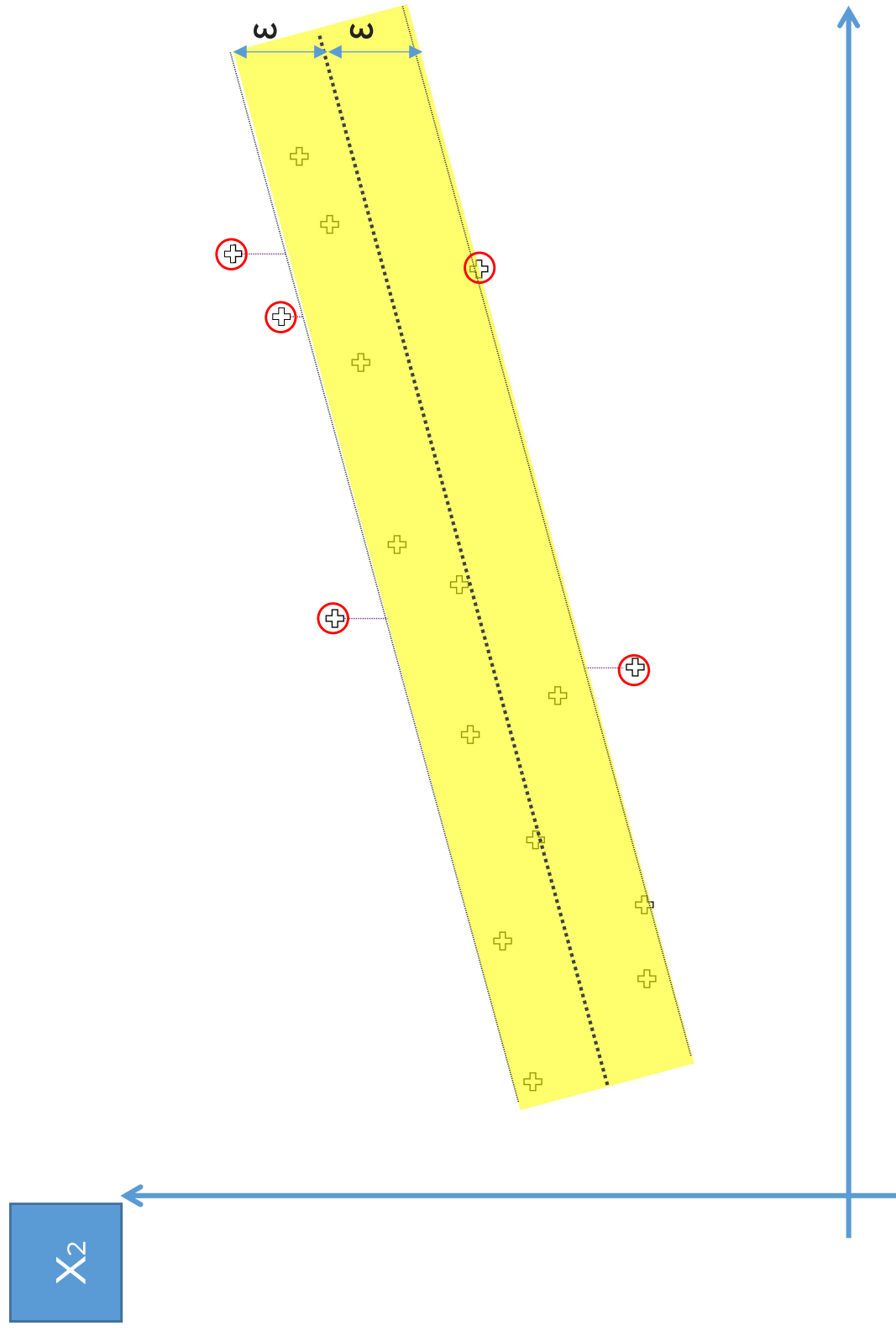
Link:

<https://core.ac.uk/download/pdf/81523322.pdf>



Heads-up about Non-Linear SVR

SVR Intuition



Python - Google Drive

support_vector_regression.ipynb

colab.research.google.com/drive/1oL2x1VBcfvEuCkNlRpTpywg6hSk81HKu#scrollTo=w_AG37TAaVqQ

Copy of support_vector_regression.ipynb

File

Edit

View

Insert

Runtime

Tools

Help

All changes saved

Files

Upload

Refresh

Mount Drive

sample_data

Position_Salaries.csv

Position_Salaries

Business Analysts

Junior Consultants

Senior Consultants

Manager

Country Managers

Region Managers

Partner

Senior Partner

C-level

CEO

Show 10 per page

Visualising the SVR results (for higher resolution and smoother curve)

1 X_grid = np.arange(min(sc_X.inverse_transform(X)), max(sc_X.inverse_transform(X)), 0.1)

2 X_grid = X_grid.reshape((len(X_grid), 1))

3 plt.scatter(sc_X.inverse_transform(X), sc_y.inverse_transform(y), color = 'red')

4 plt.plot(X_grid, sc_y.inverse_transform(regressor.predict(sc_X.transform(X_grid))), color = 'blue')

5 plt.title('Truth or Bluff (Support Vector Regression)')

6 plt.xlabel('Position level')

7 plt.ylabel('Salary')

8 plt.show()

Disk

76.87 GB available

Heads-up about Non-Linear SVM

Section on SVM:

- SVM Intuition

Section on Kernel SVM:

- Kernel SVM Intuition
- Mapping to a higher dimension
- The Kernel Trick
- Types of Kernel Functions
- Non-linear Kernel SVM

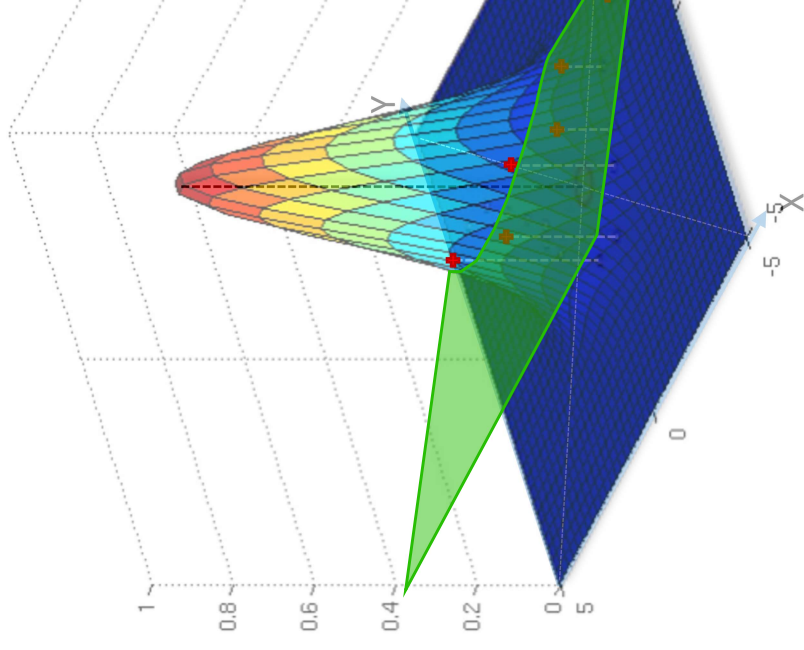


Image source: <http://www.cs.toronto.edu/~duvenaud/>

Decision Tree Intuition

Decision Tree Intuition

CART

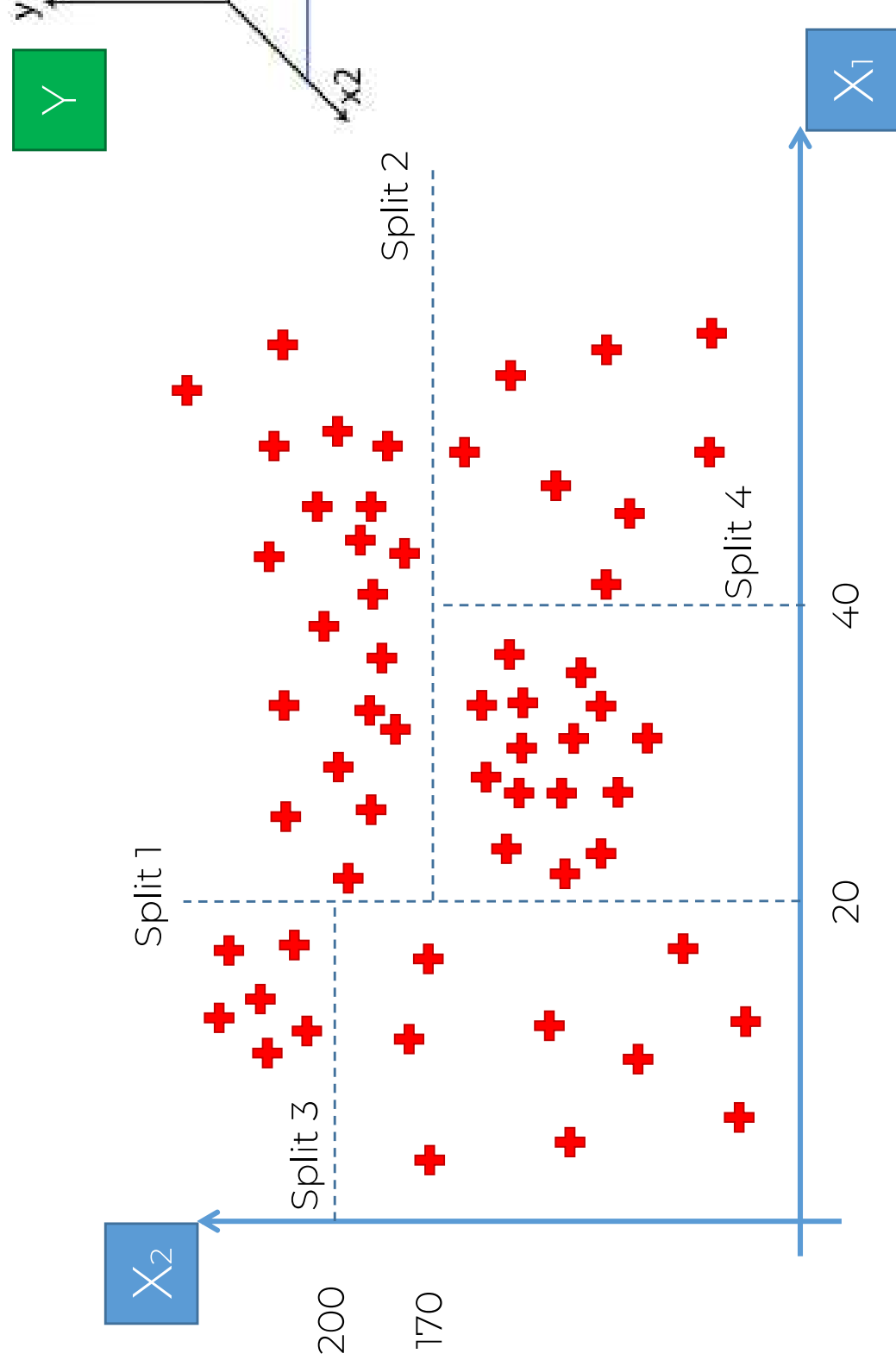


Classification
Trees



Regression
Trees

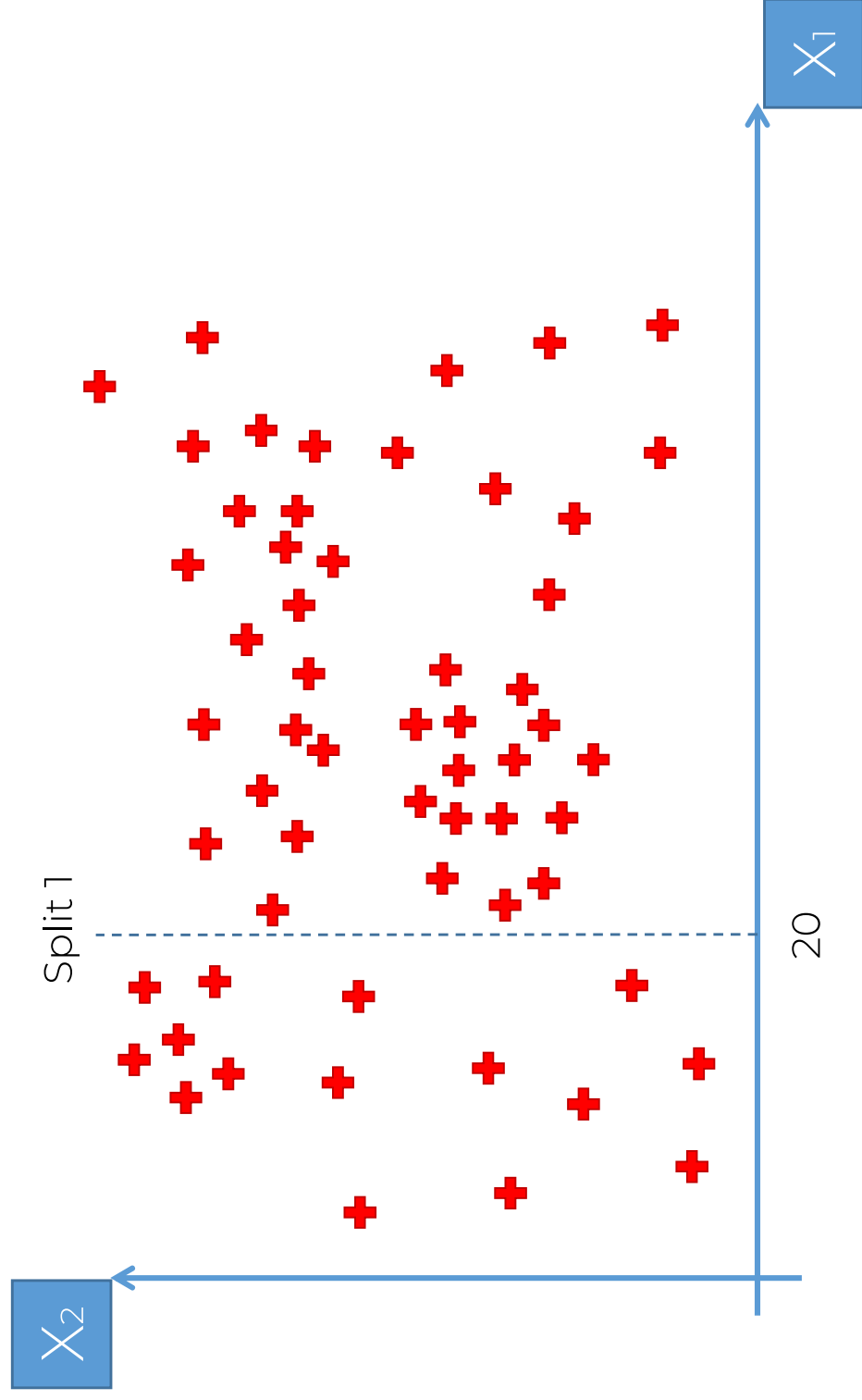
Decision Tree Intuition



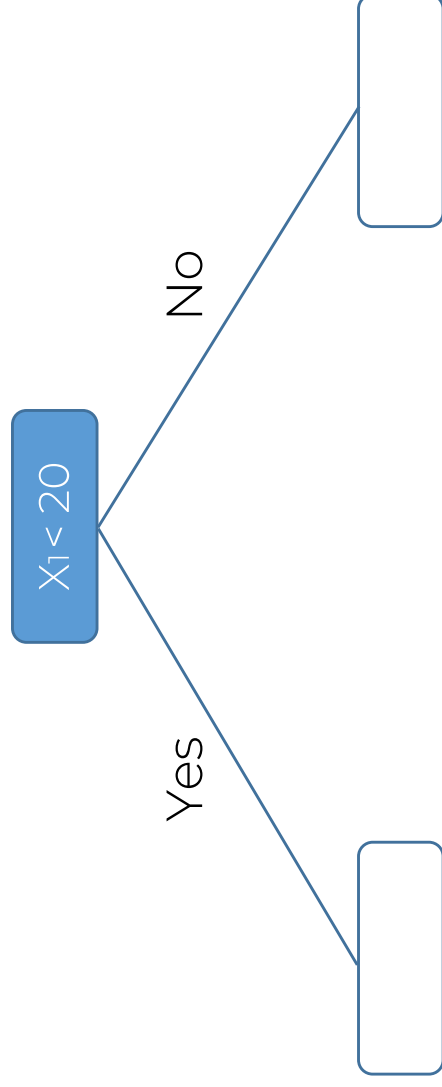
Decision Tree Intuition

Rewind...

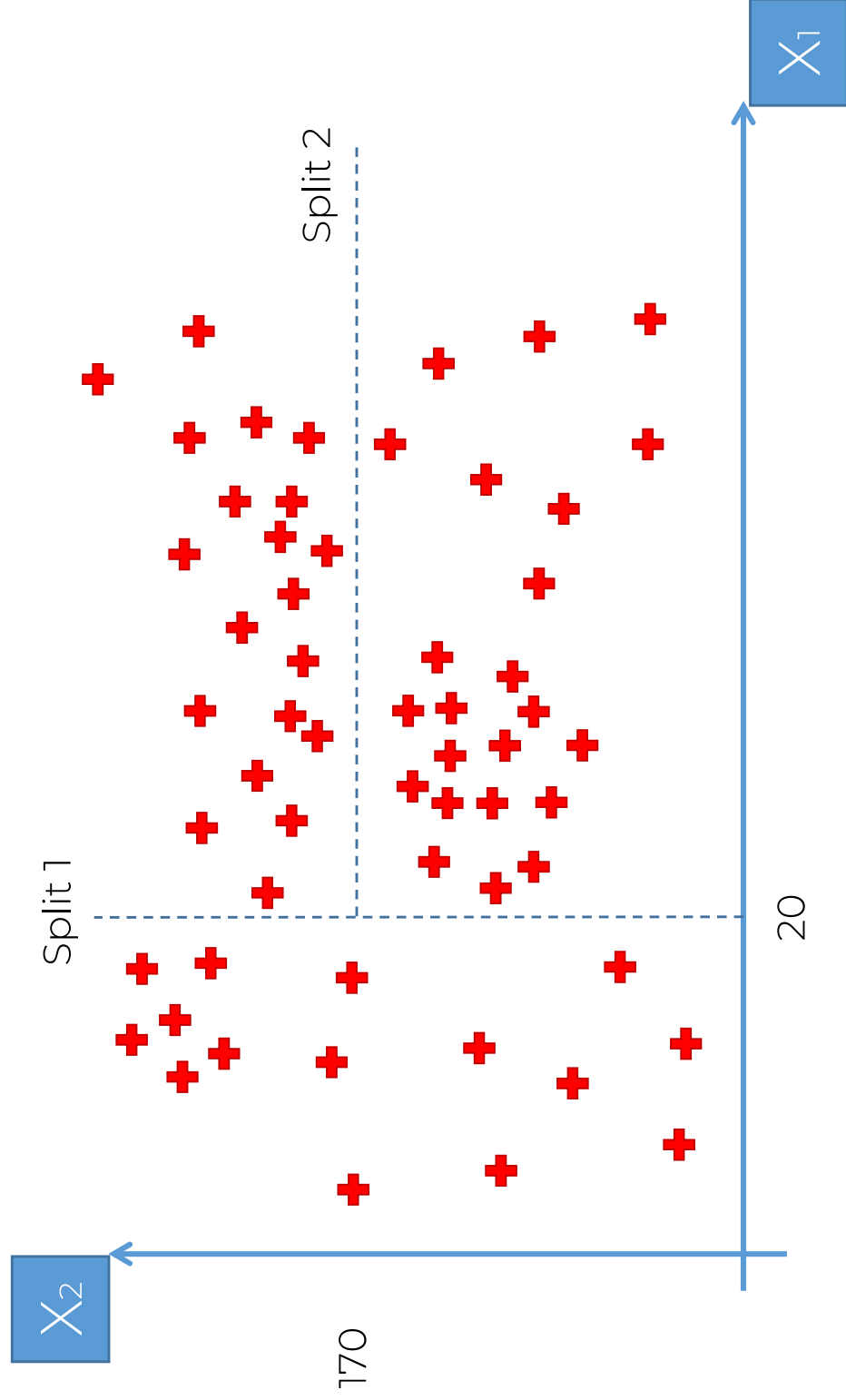
Decision Tree Intuition



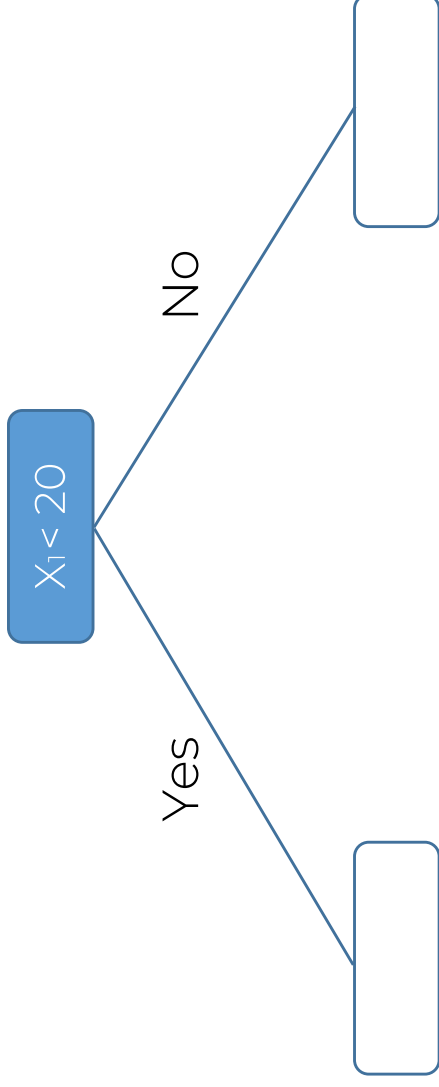
Decision Tree Intuition



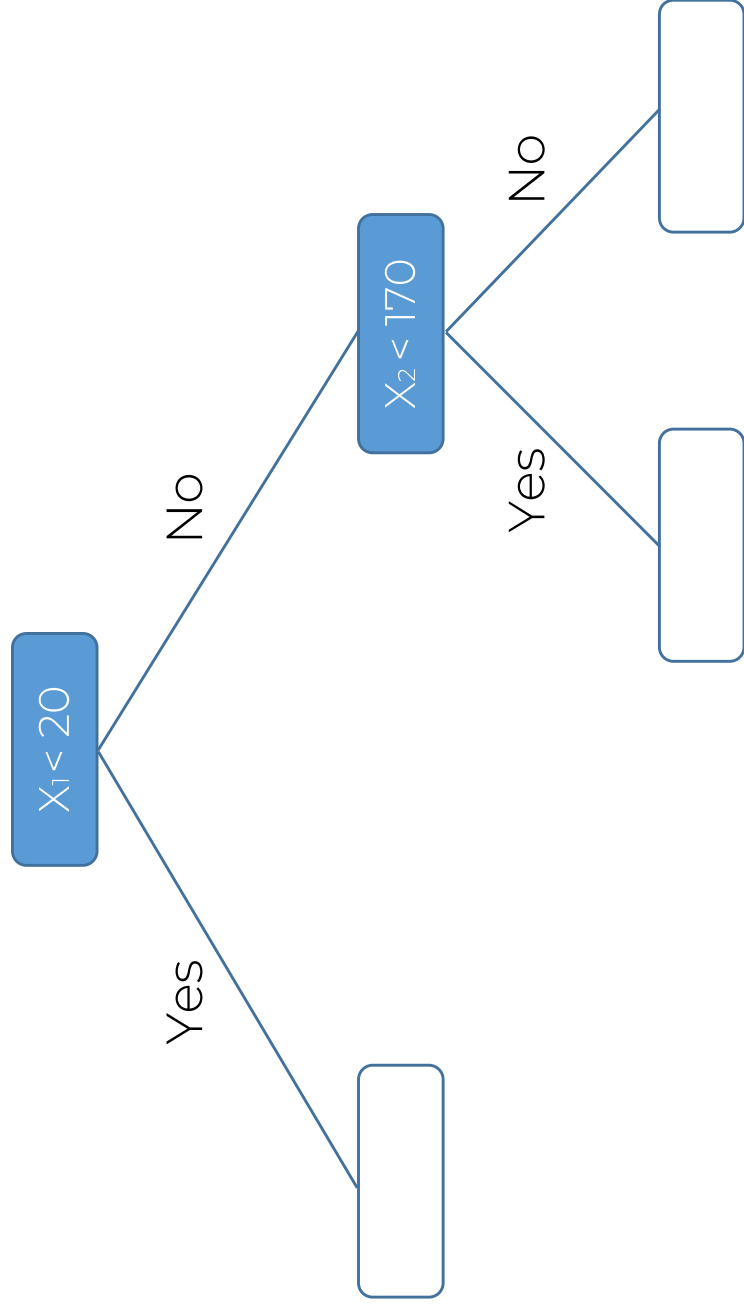
Decision Tree Intuition



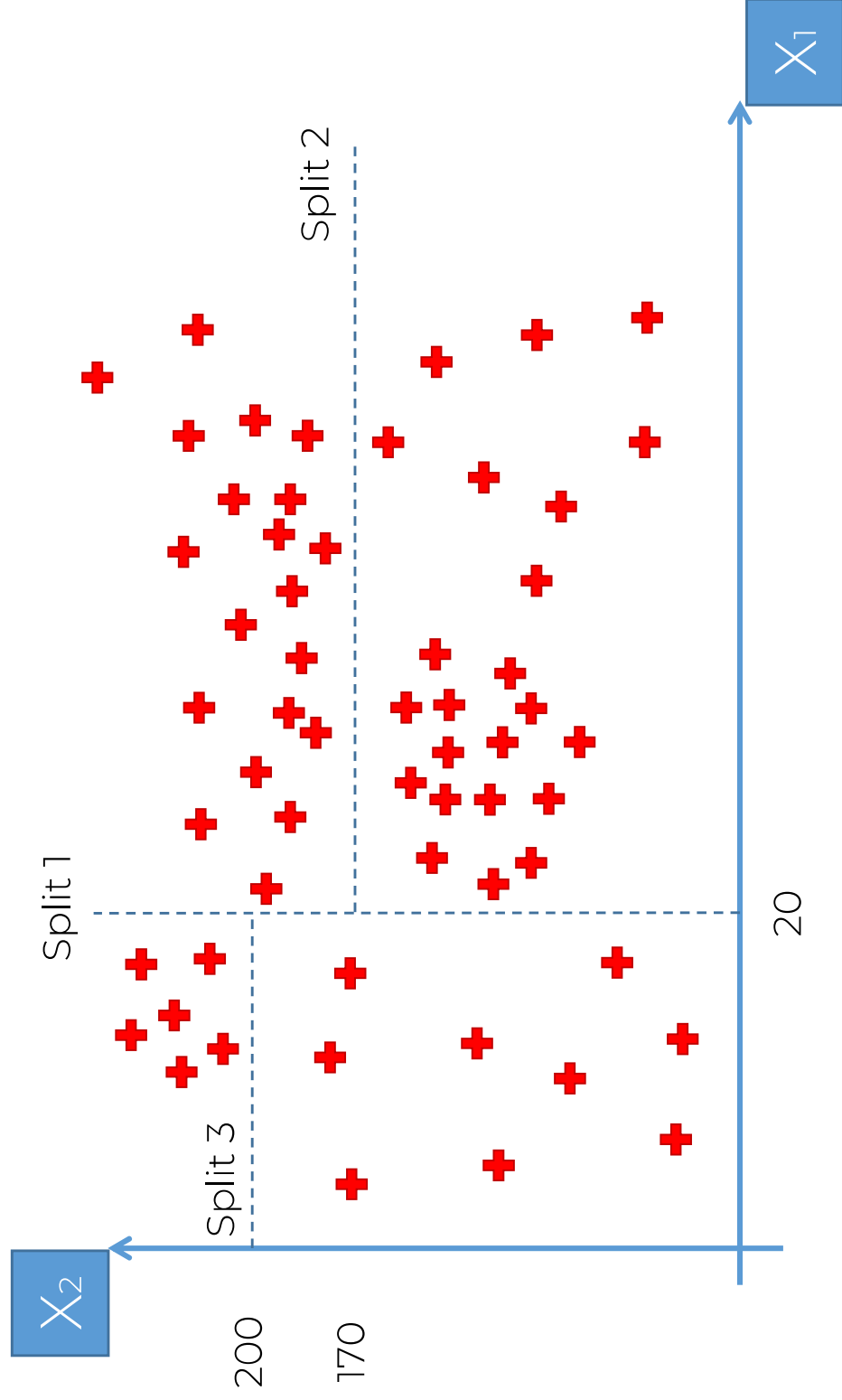
Decision Tree Intuition



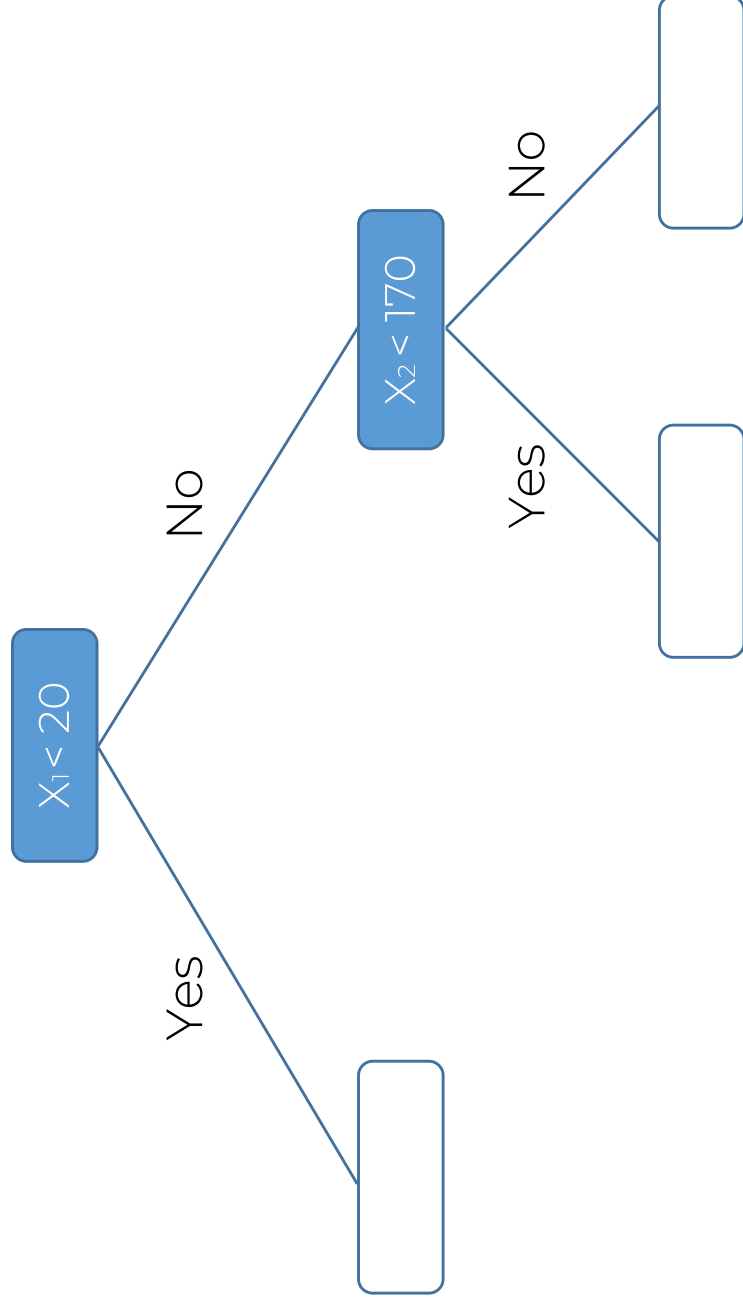
Decision Tree Intuition



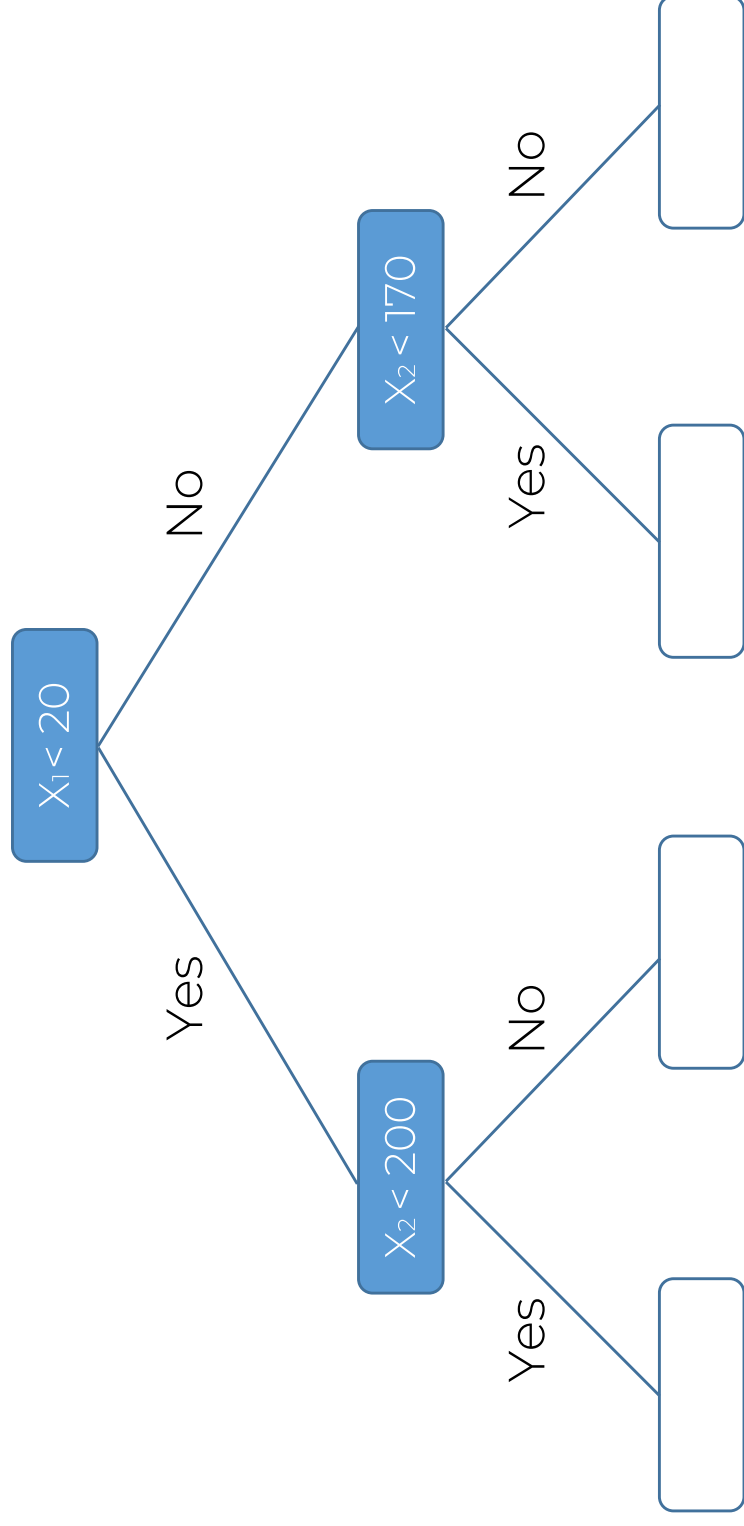
Decision Tree Intuition



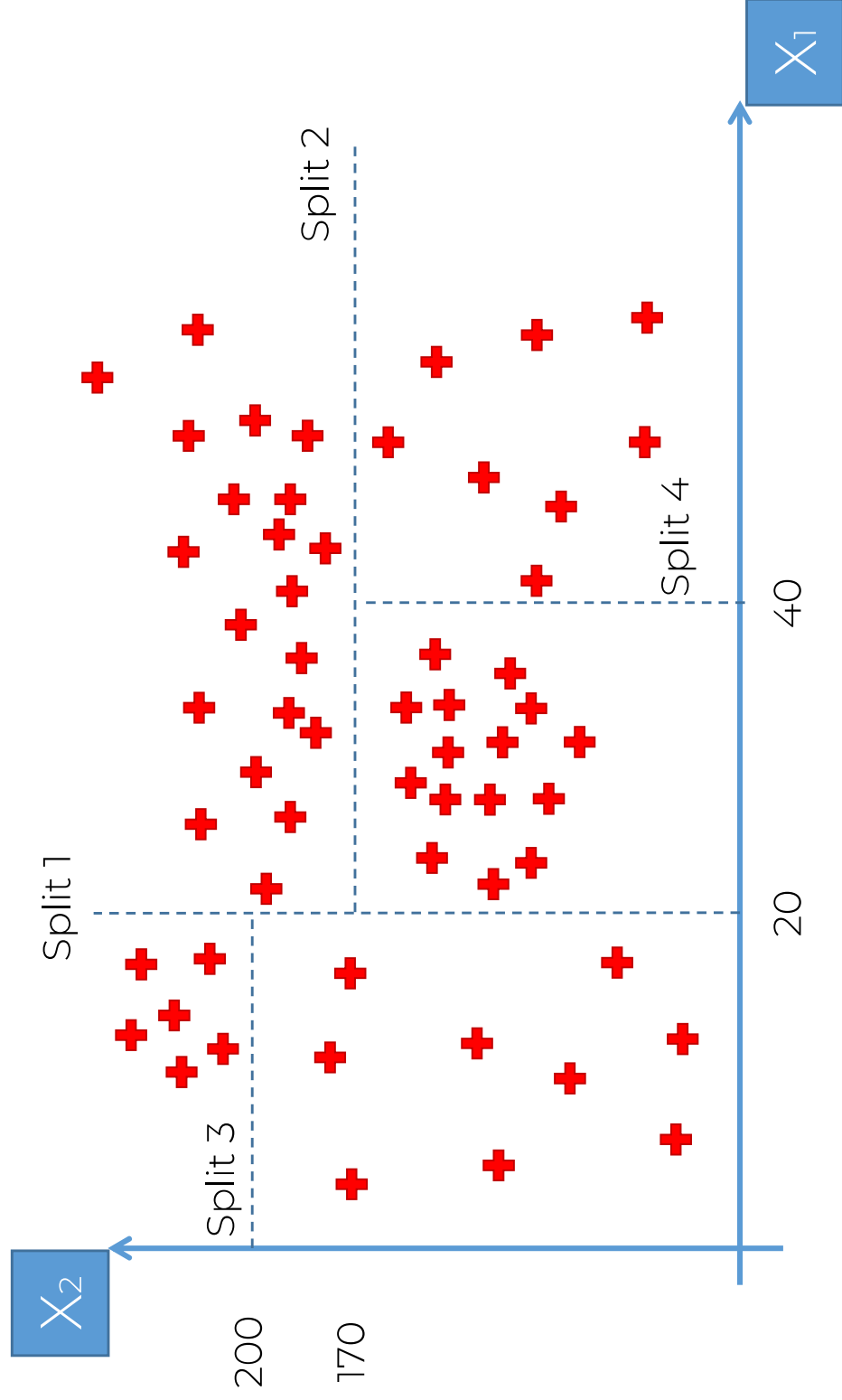
Decision Tree Intuition



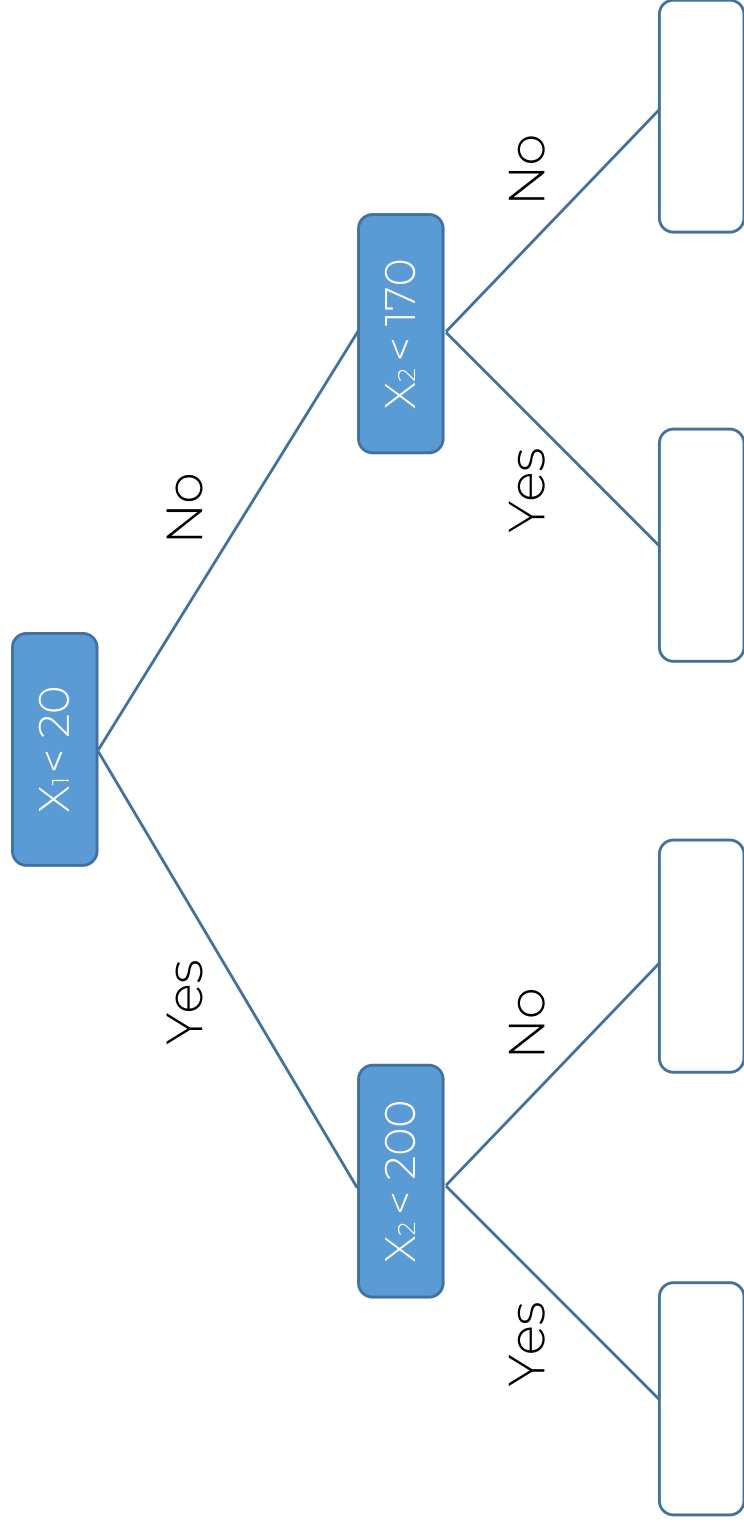
Decision Tree Intuition



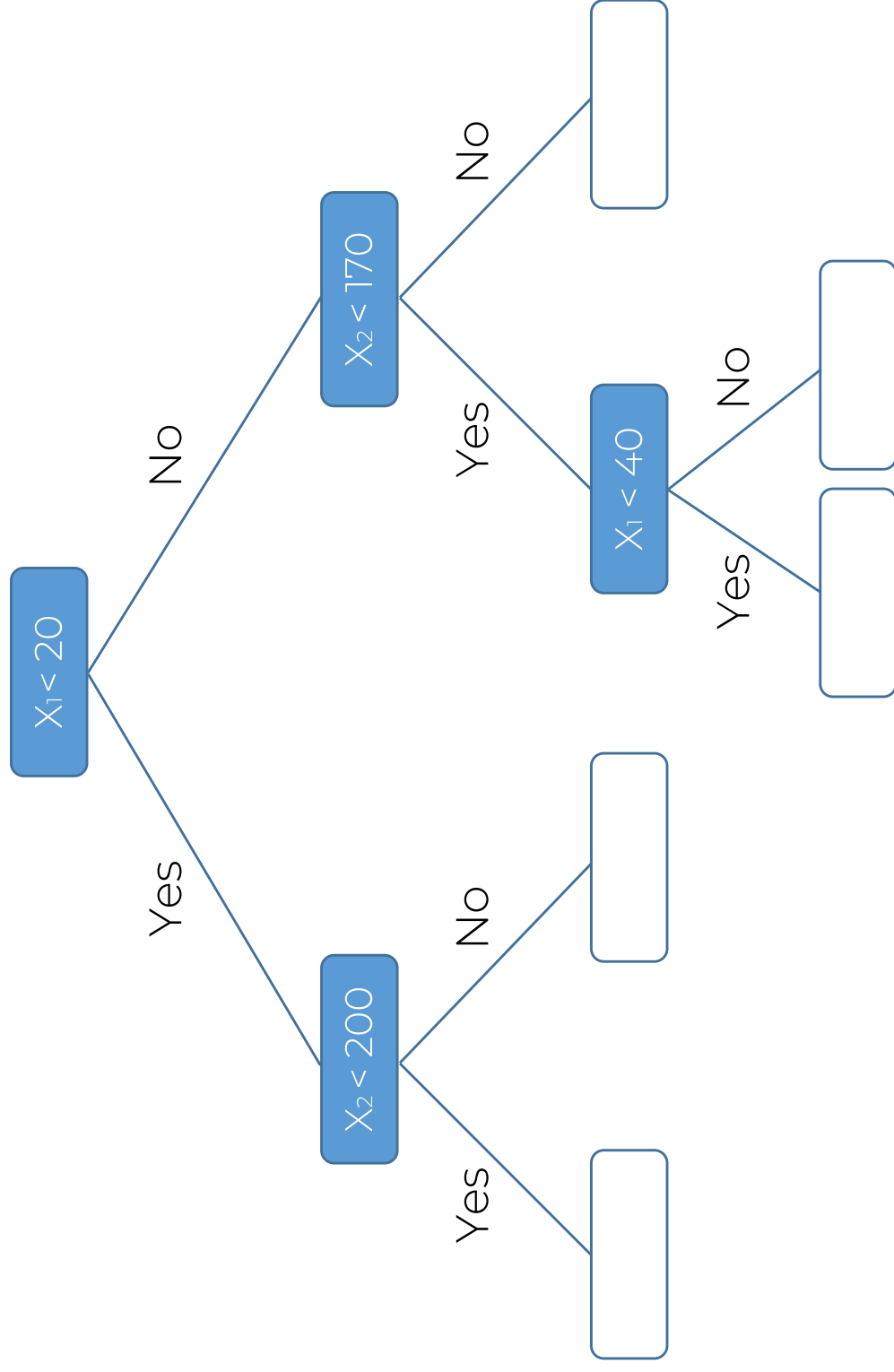
Decision Tree Intuition



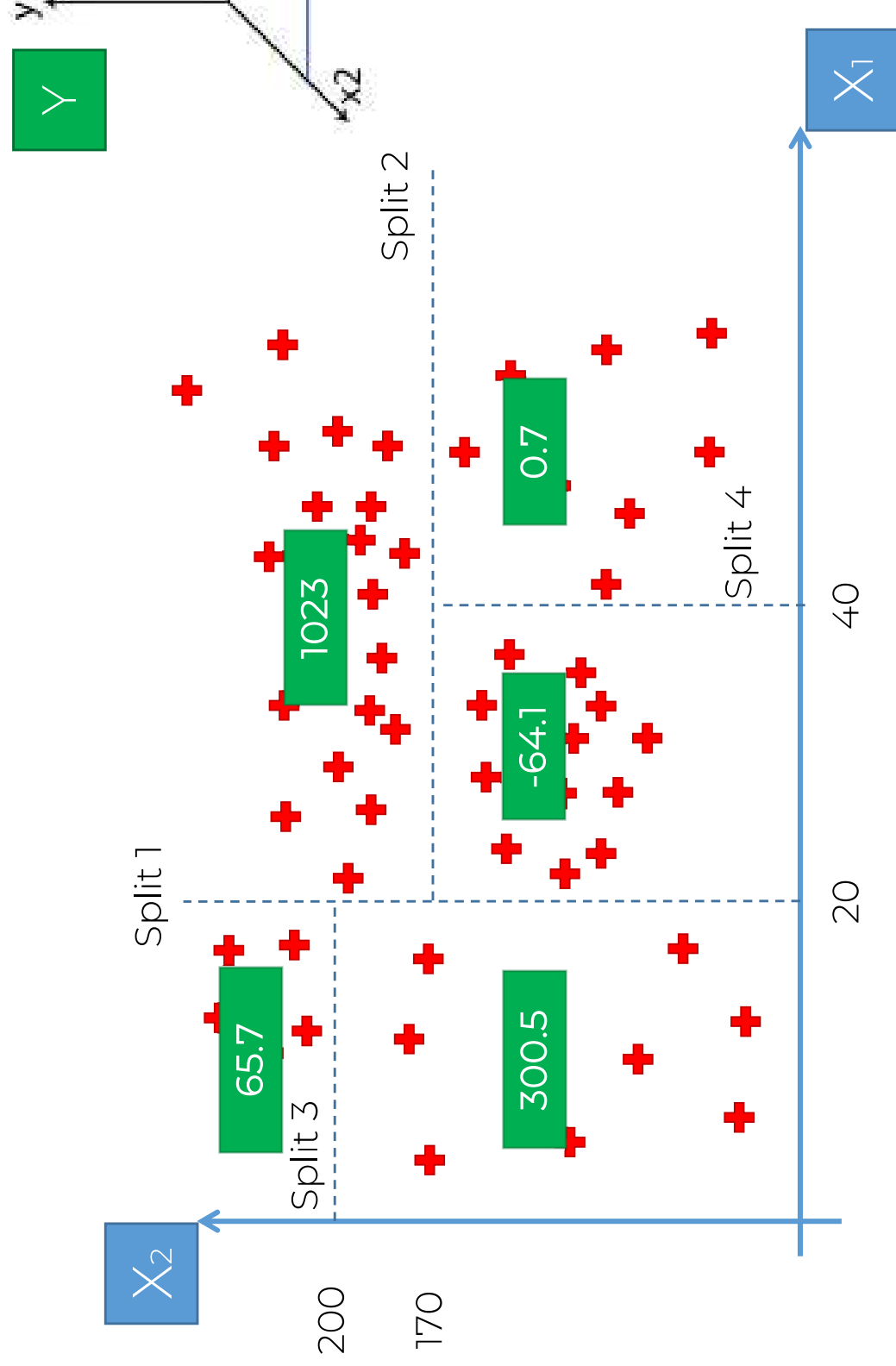
Decision Tree Intuition



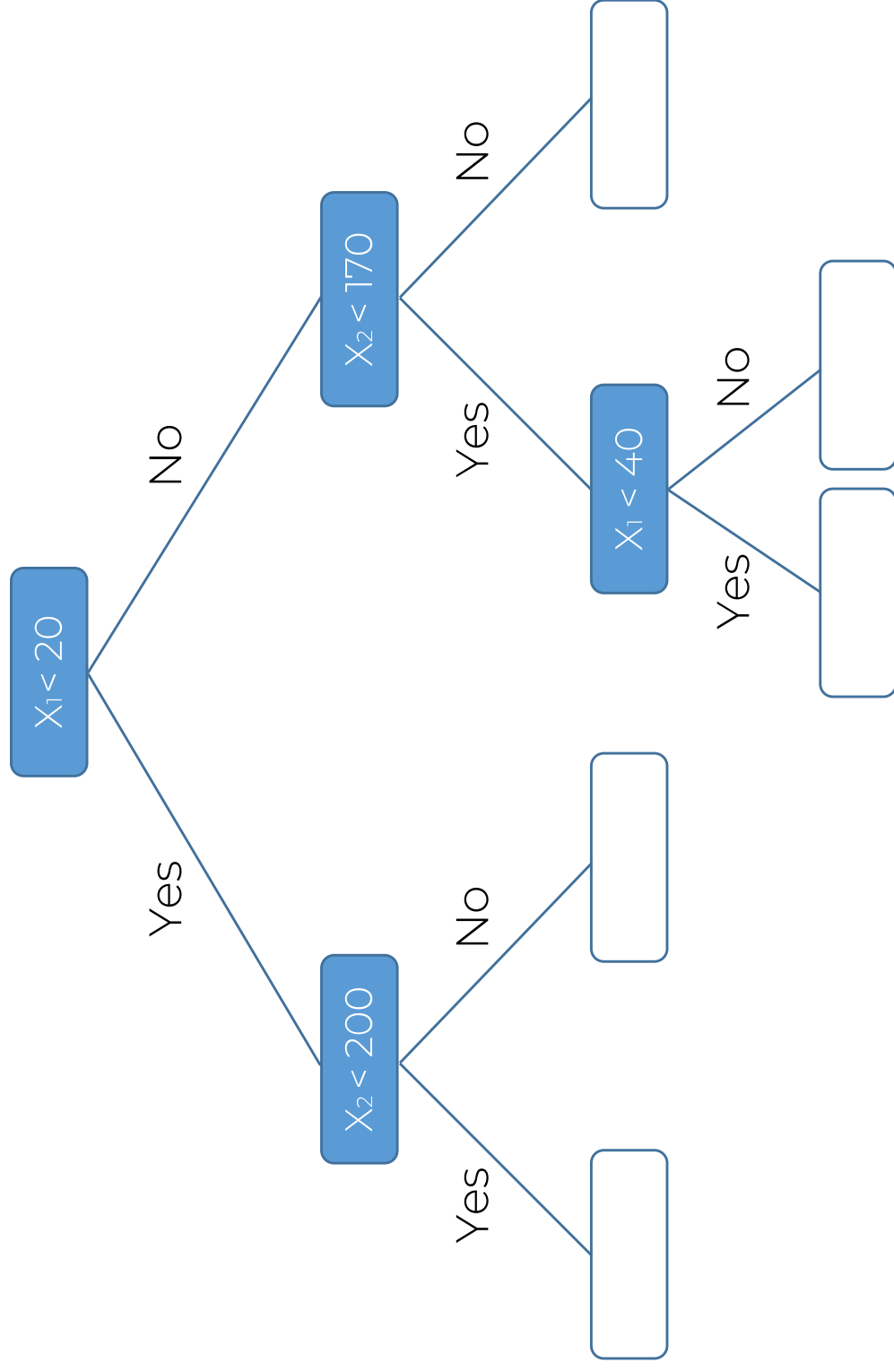
Decision Tree Intuition



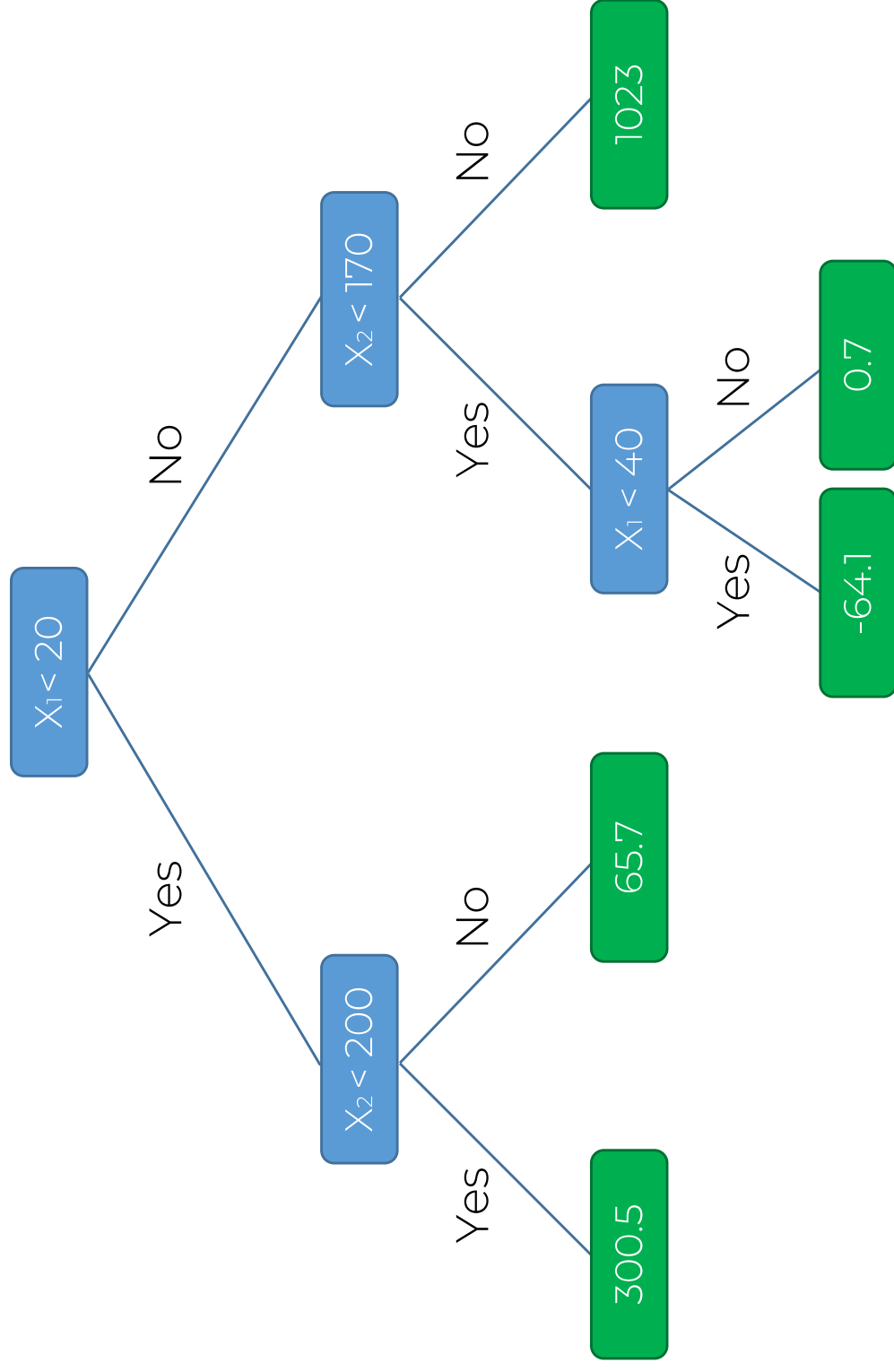
Decision Tree Intuition



Decision Tree Intuition



Decision Tree Intuition



Random Forest Intuition

Random Forest Intuition

Ensemble Learning

Random Forest Intuition

STEP 1: Pick at random K data points from the Training set.



STEP 2: Build the Decision Tree associated to these K data points.



STEP 3: Choose the number N_{tree} of trees you want to build and repeat ST



STEP 4: For a new data point, make each one of your N_{tree} trees predict the value for the data point in question, and assign the new data point the average of the predicted Y values.

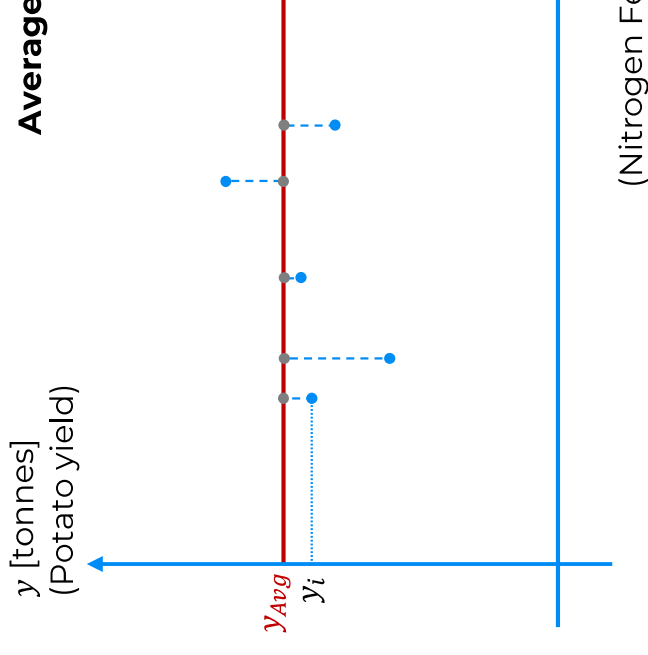
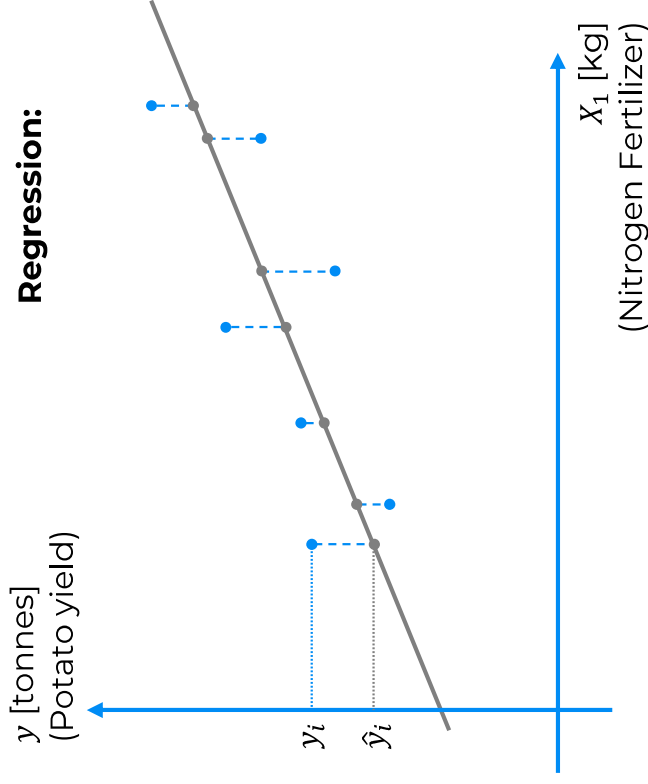


R Squared

© SuperDataScience



R Squared



$$SS_{res} = SUM(y_i - \hat{y}_i)^2$$

$$SS_{tot} = SUM(y_i - y_{Avg})^2$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Rule of thumb (for our case)

- 1.0 = Perfect fit (suspicious)
- ~0.9 = Very good
- <0.7 = Not great
- <0.4 = Terrible
- <0 = Model makes no sense

*This is highly dependent on the context



Adjusted R Squared

© SuperDataScience



Adjusted R Squared

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

R^2 – Goodness of fit
(greater is better)

Problem:

$$\hat{y} = b_0 + b_1X_1 + b_2X_2 + \boxed{+ b_3X_3}$$

SS_{tot} doesn't change

SS_{res} will decrease or stay the same

(This is because of Ordinary Least Squares: SS_{res})

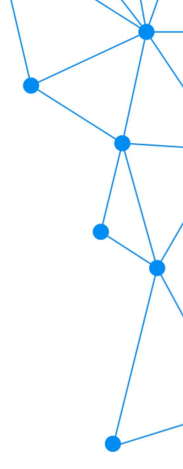
Solution:

$$Adj R^2 = 1 - (1 - R^2) \times \frac{n - 1}{n - k - 1}$$

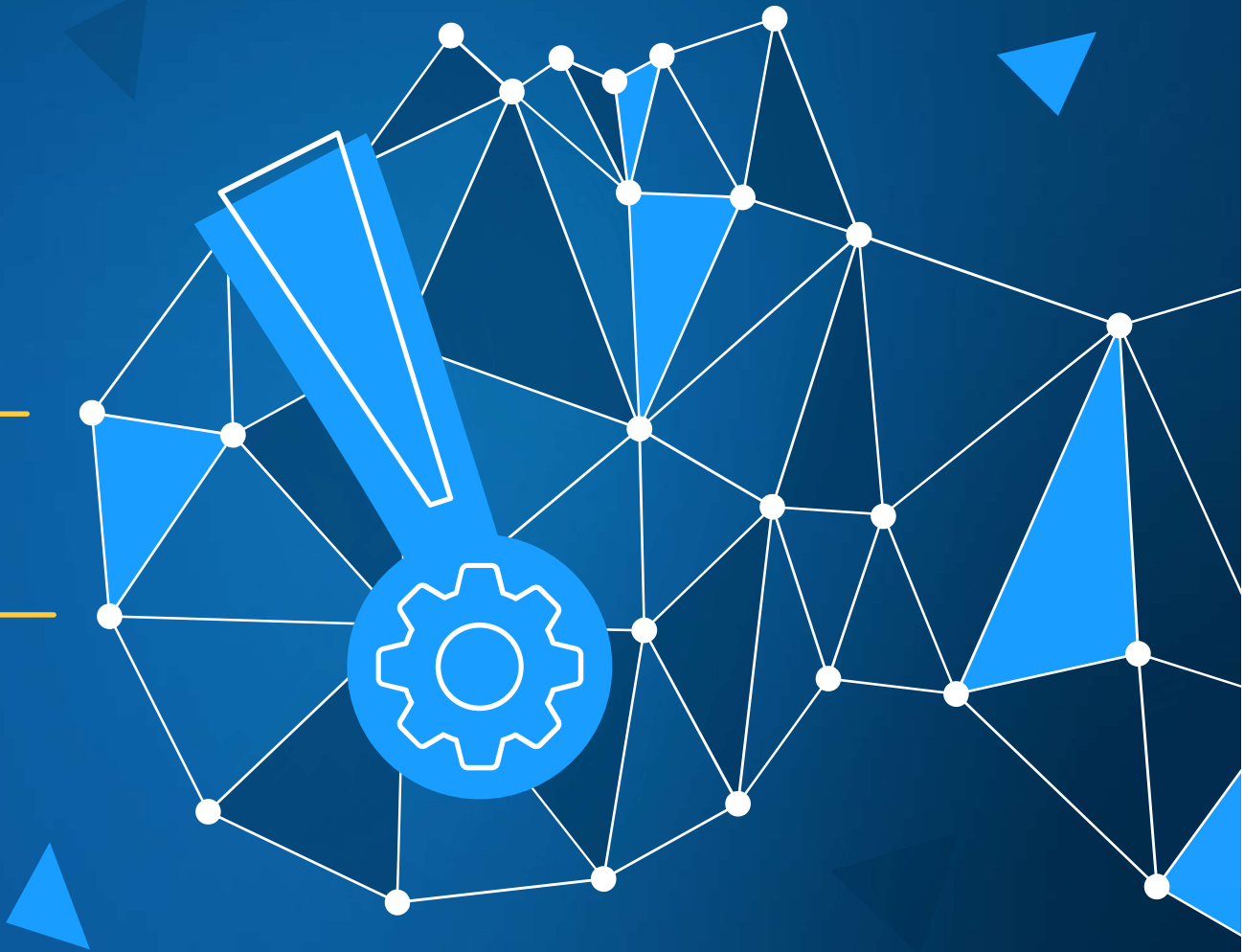
k – number of independent variables

n – sample size

$$SS_{res} = SUM(\overset{\text{orange arrow}}{y_i} - \hat{y}_i)^2$$

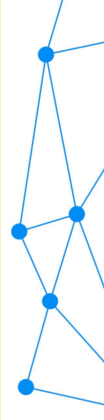
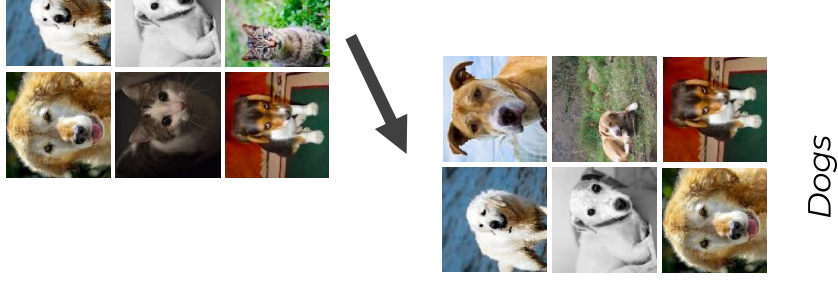
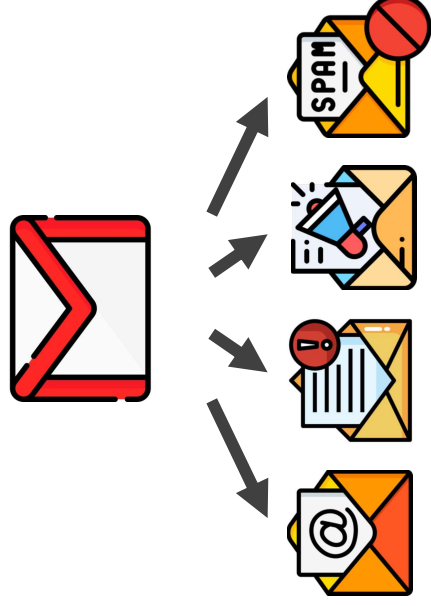
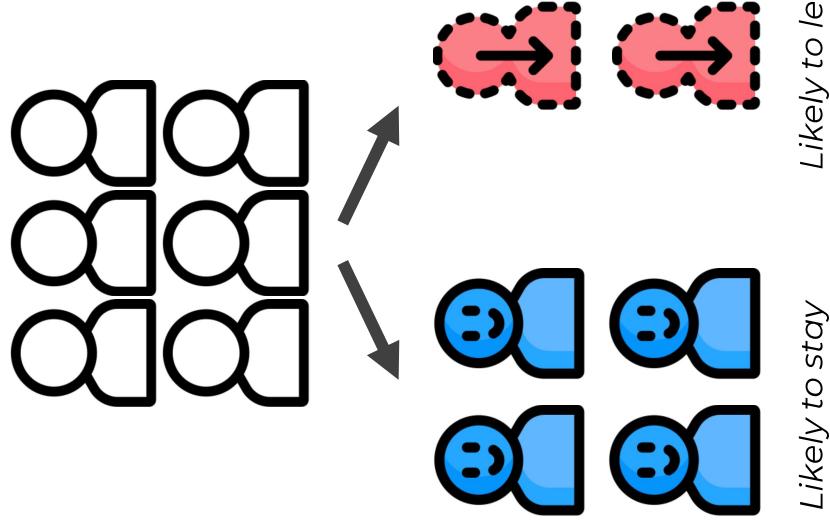


Classification

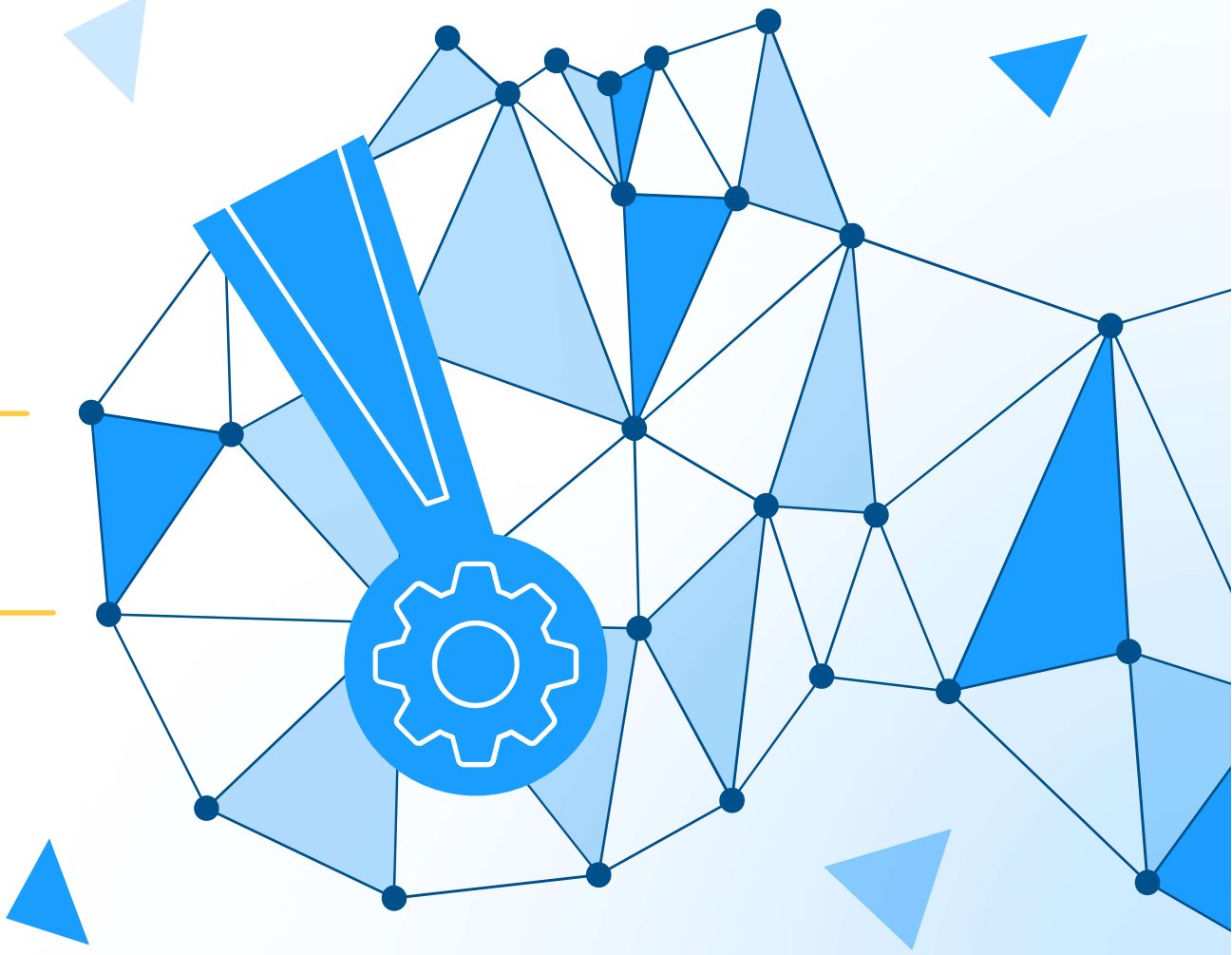


What is Classification?

Classification: a Machine Learning technique to identify the category of new observations based on training data.

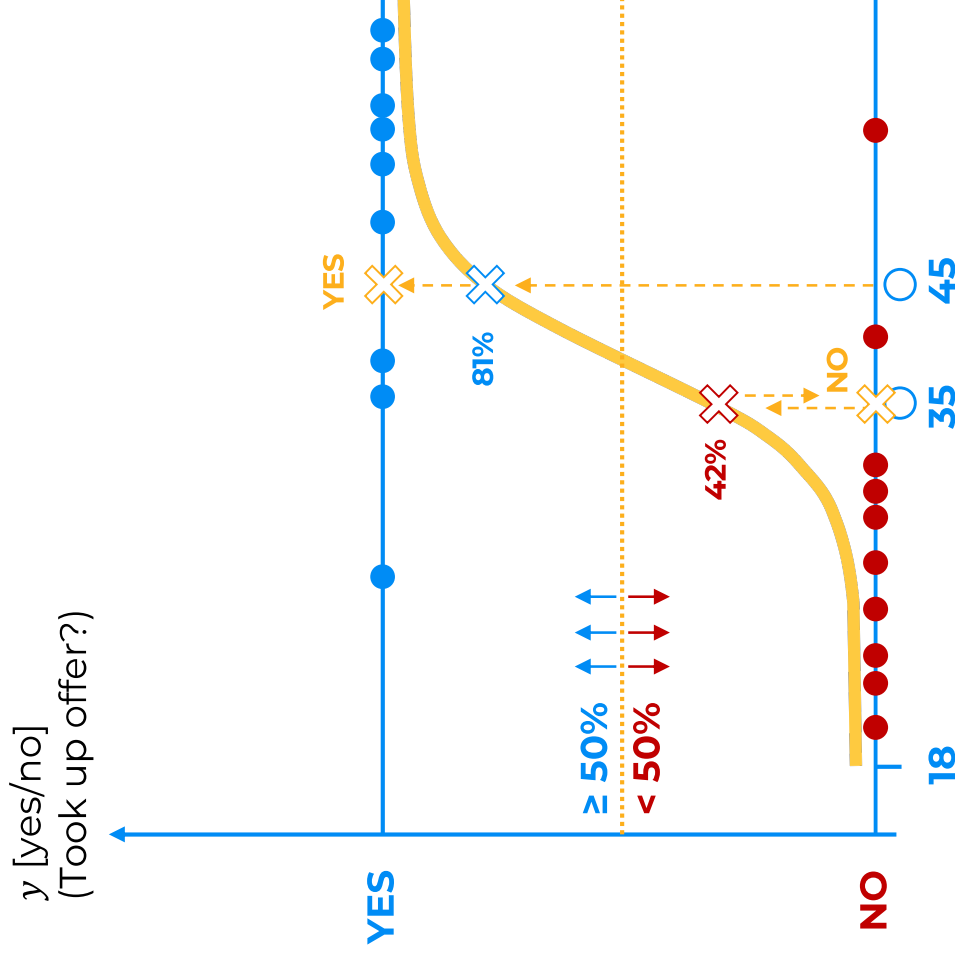
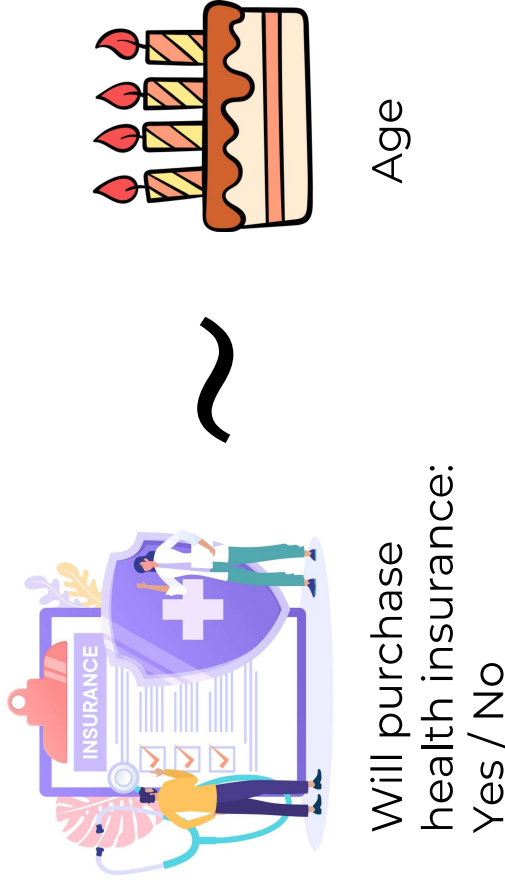


Logistic Regression



Logistic Regression

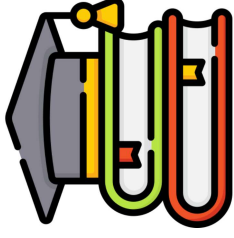
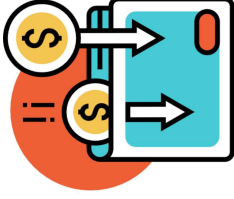
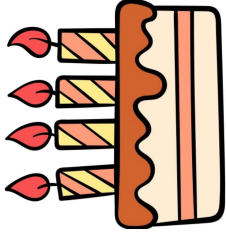
Logistic regression: predict a categorical dependent variable from a number of independent variables.



Logistic Regression



~



Will purchase
health insurance:
Yes / No

Age

Income

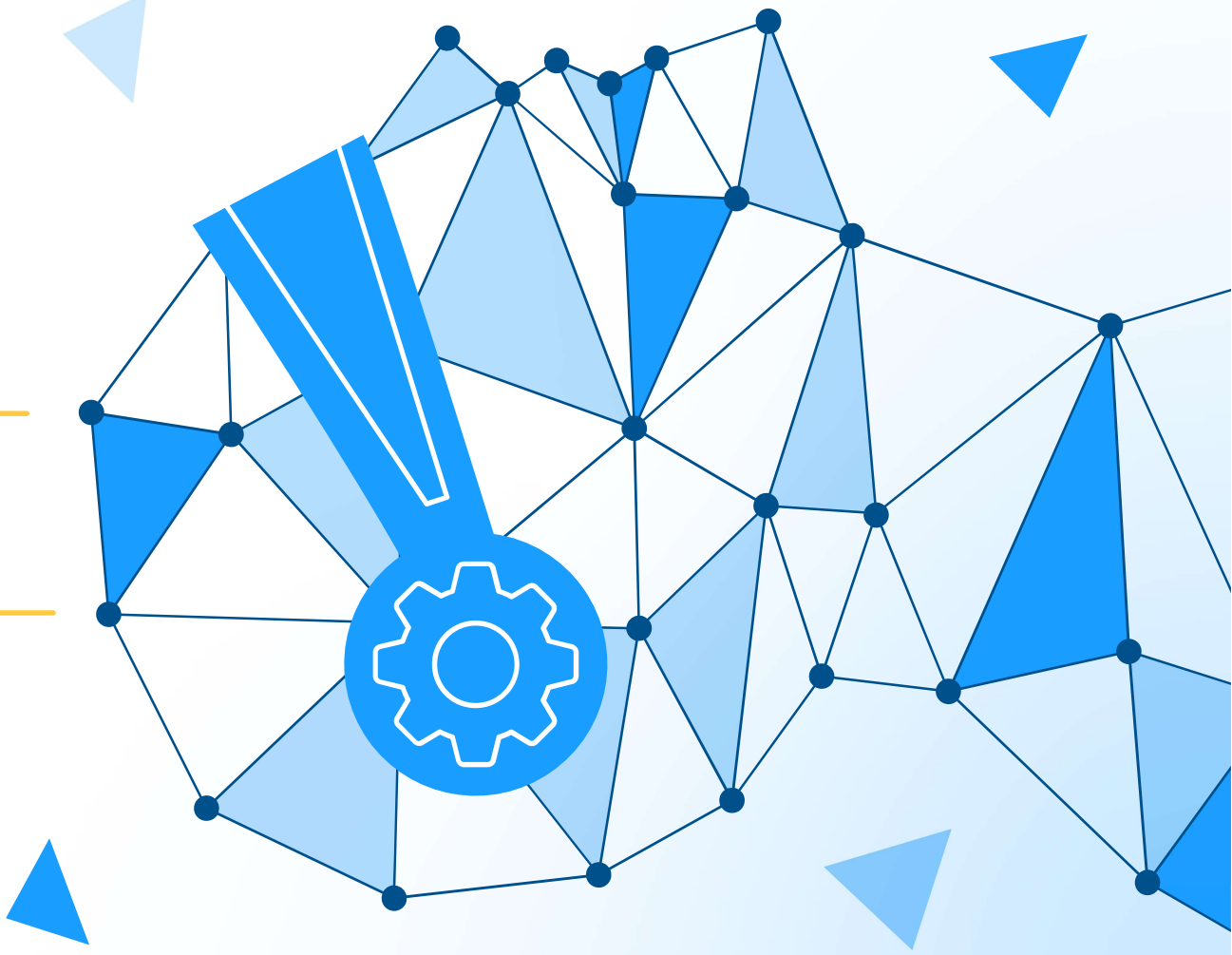
Level of
Education

Family
Size

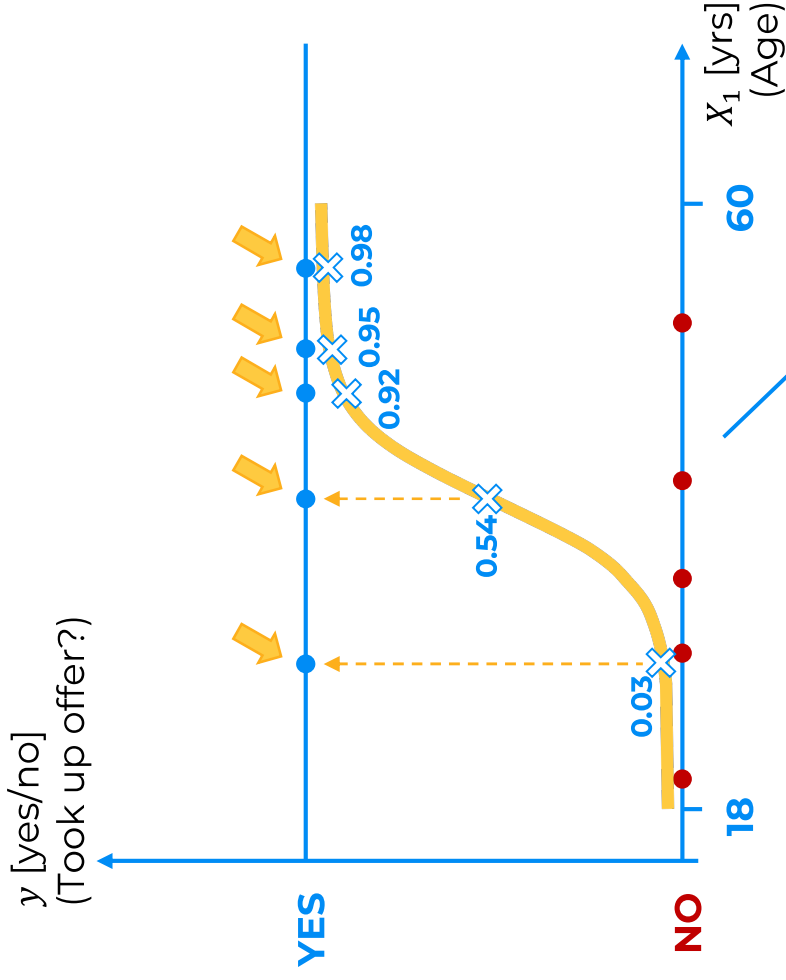
$$\ln \frac{p}{1-p} = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4$$



Maximum Likelihood

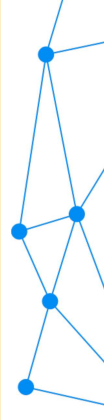
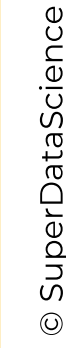
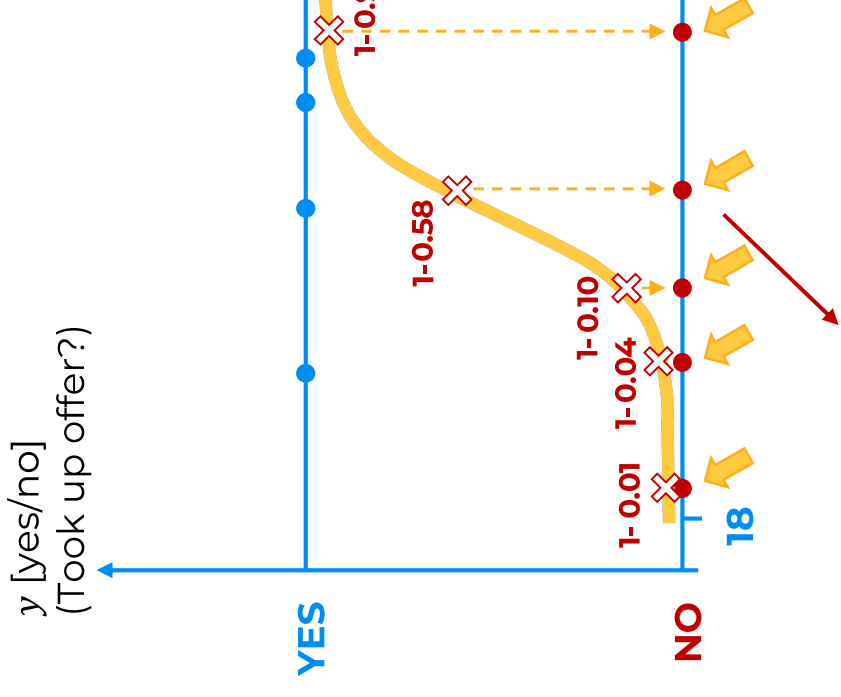


Maximum Likelihood

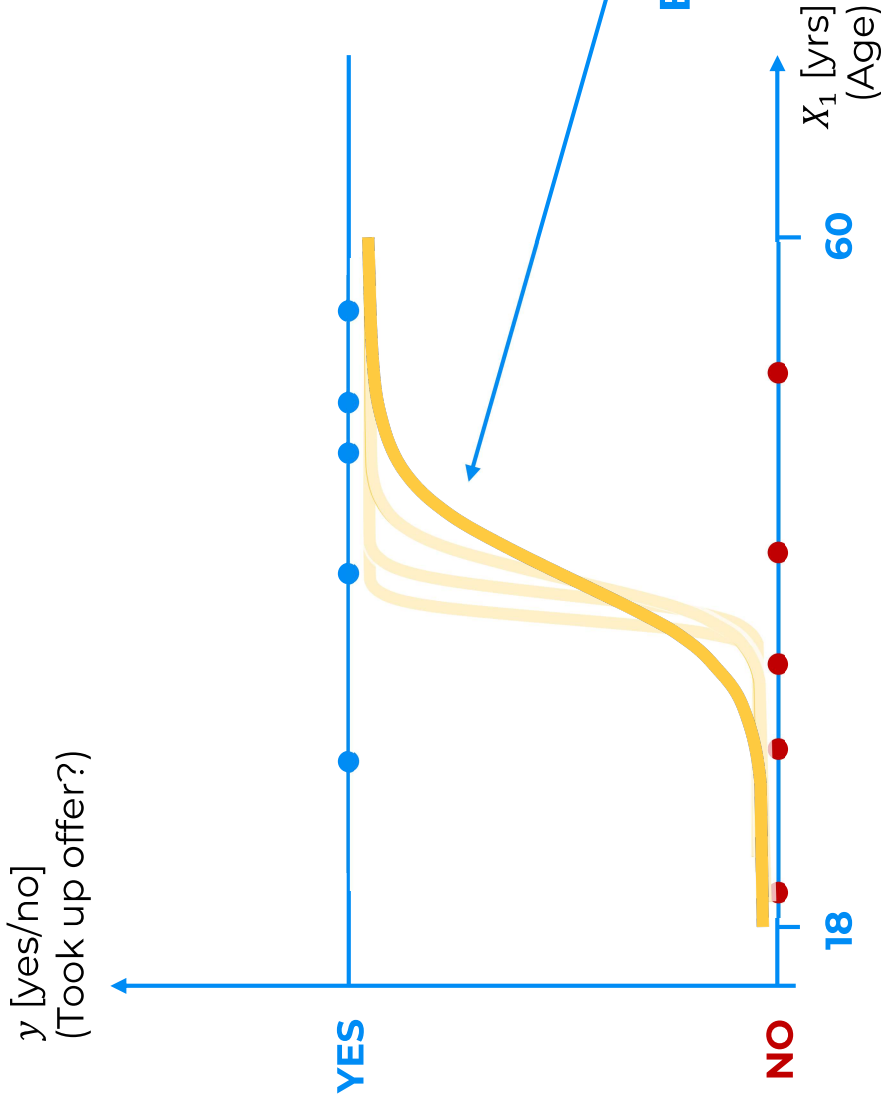


$$\text{Likelihood} = 0.03 \times 0.54 \times 0.92 \times 0.95 \times 0.98 \times (1 - 0.01) \times (1 - 0.04) \times (1 - 0.10) \times (1 - 0.58) \times (1 - 0.01)$$

Likelihood = 0.00019939



Maximum Likelihood



Likelihood = 0.000007418

Likelihood = 0.00012845

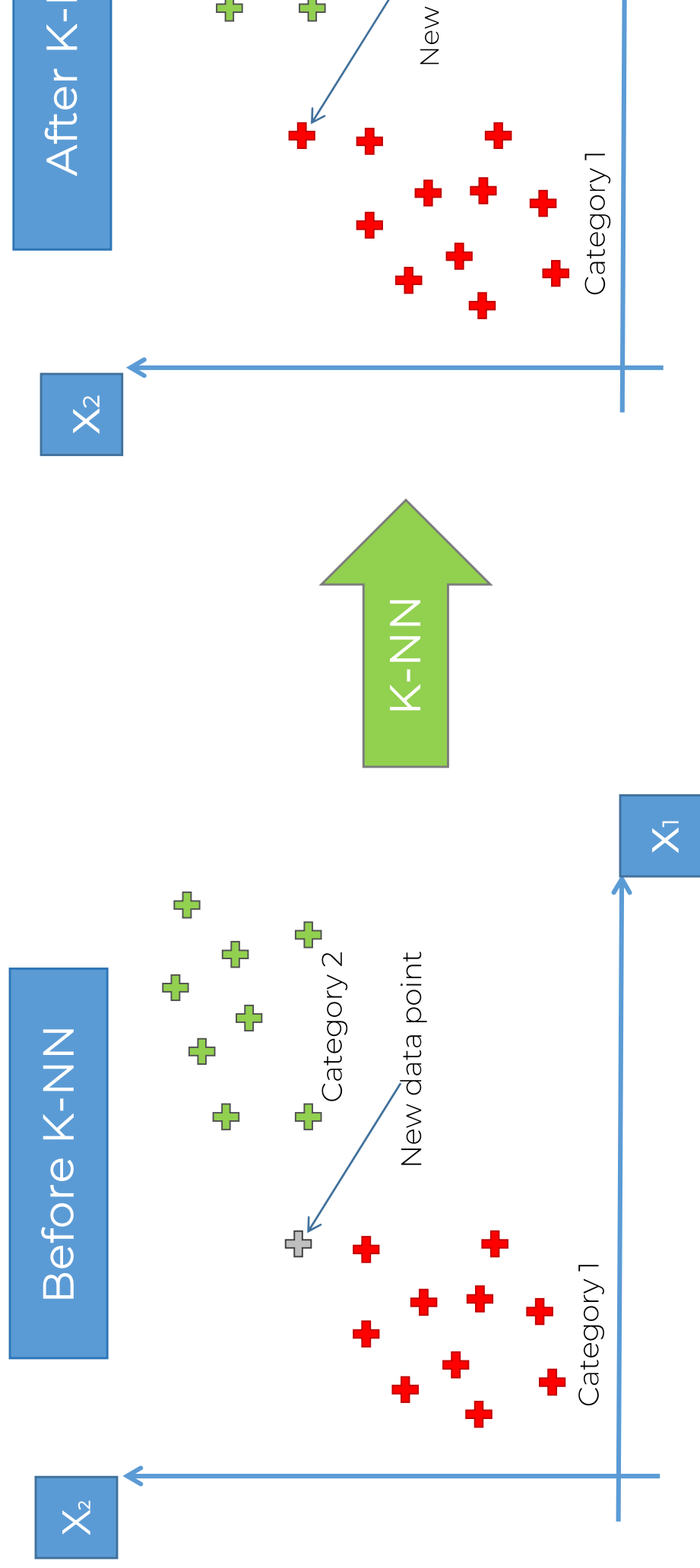
Likelihood = 0.00016553

Likelihood = 0.00019939



K-NN Intuition

What K-NN does for you



How did it do that ?

STEP 1: Choose the number K of neighbors



STEP 2: Take the K nearest neighbors of the new data point, according to the Euclidean distance



STEP 3: Among these K neighbors, count the number of data points in each category



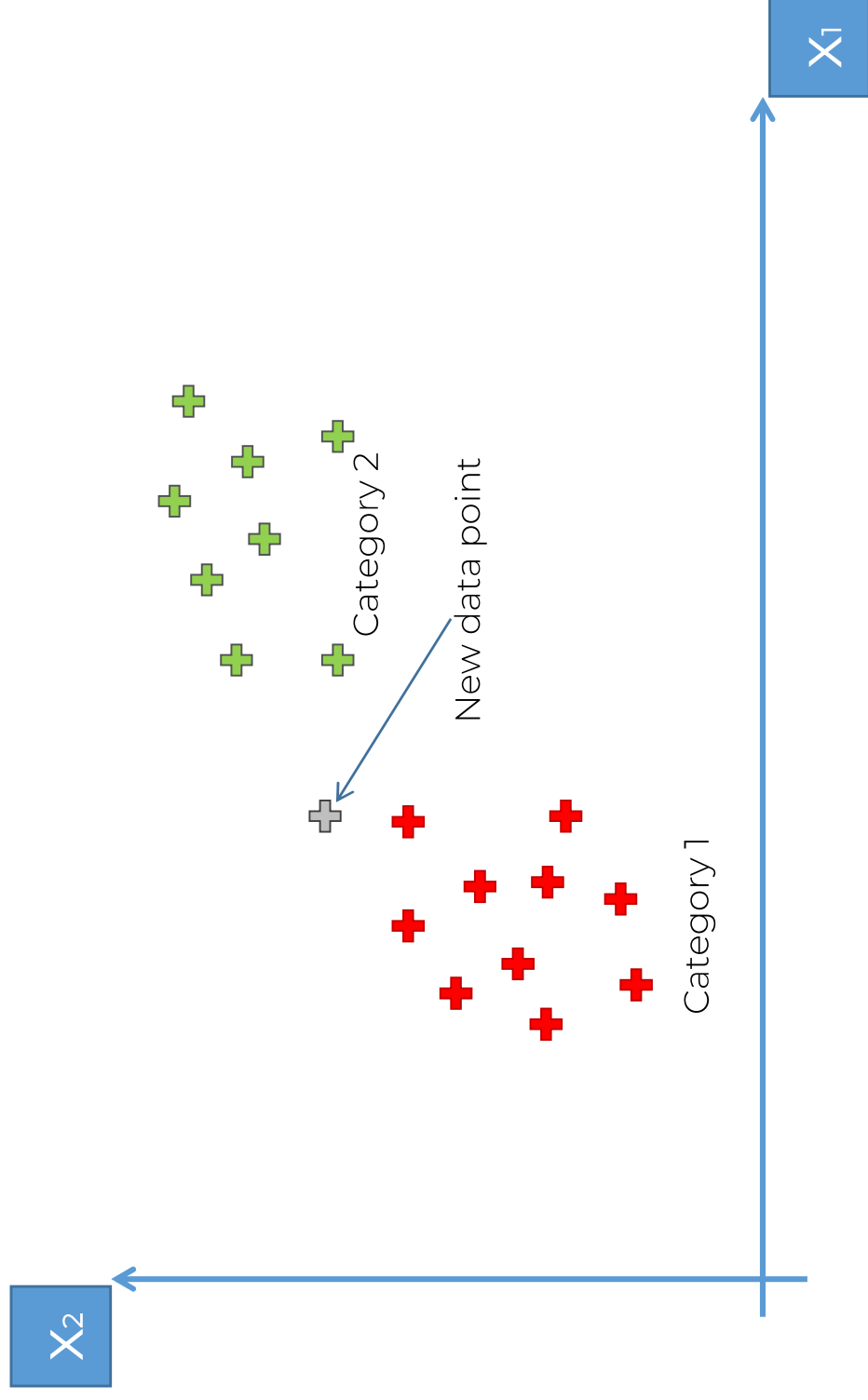
STEP 4: Assign the new data point to the category where you counted the most neighbors



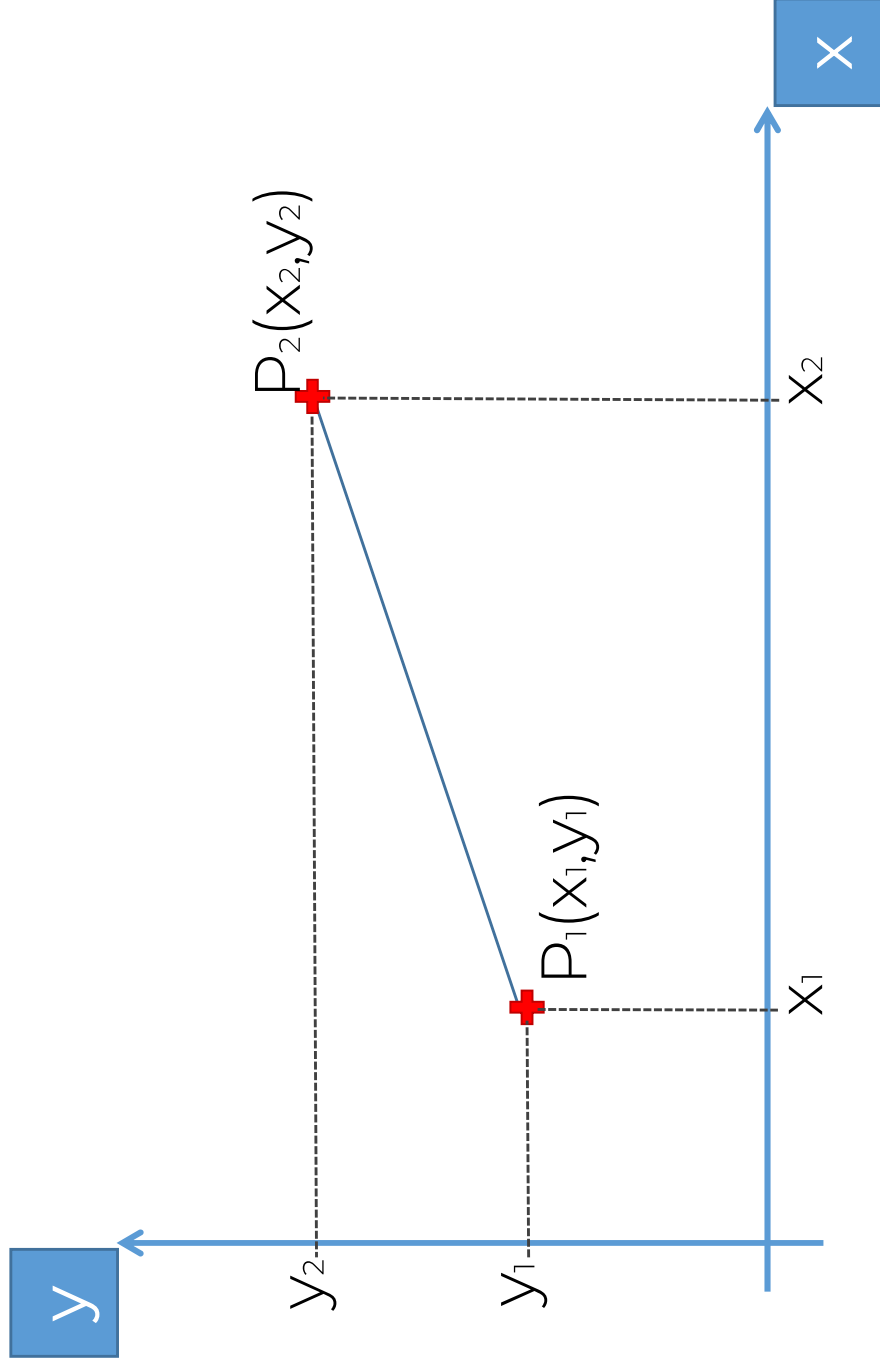
Your Model is Ready

K-NN algorithm

STEP 1: Choose the number K of neighbors: $K = 5$

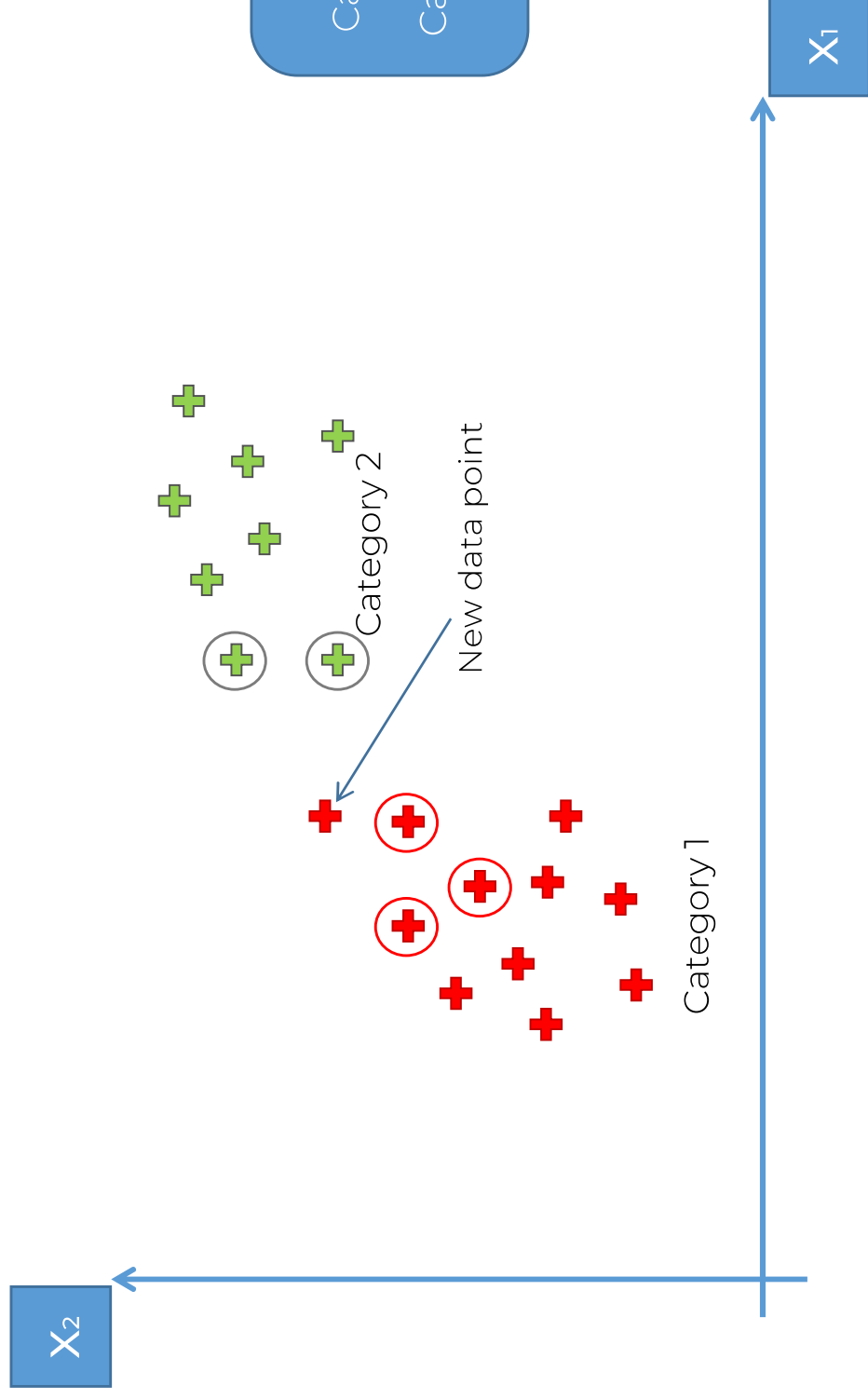


Euclidean Distance



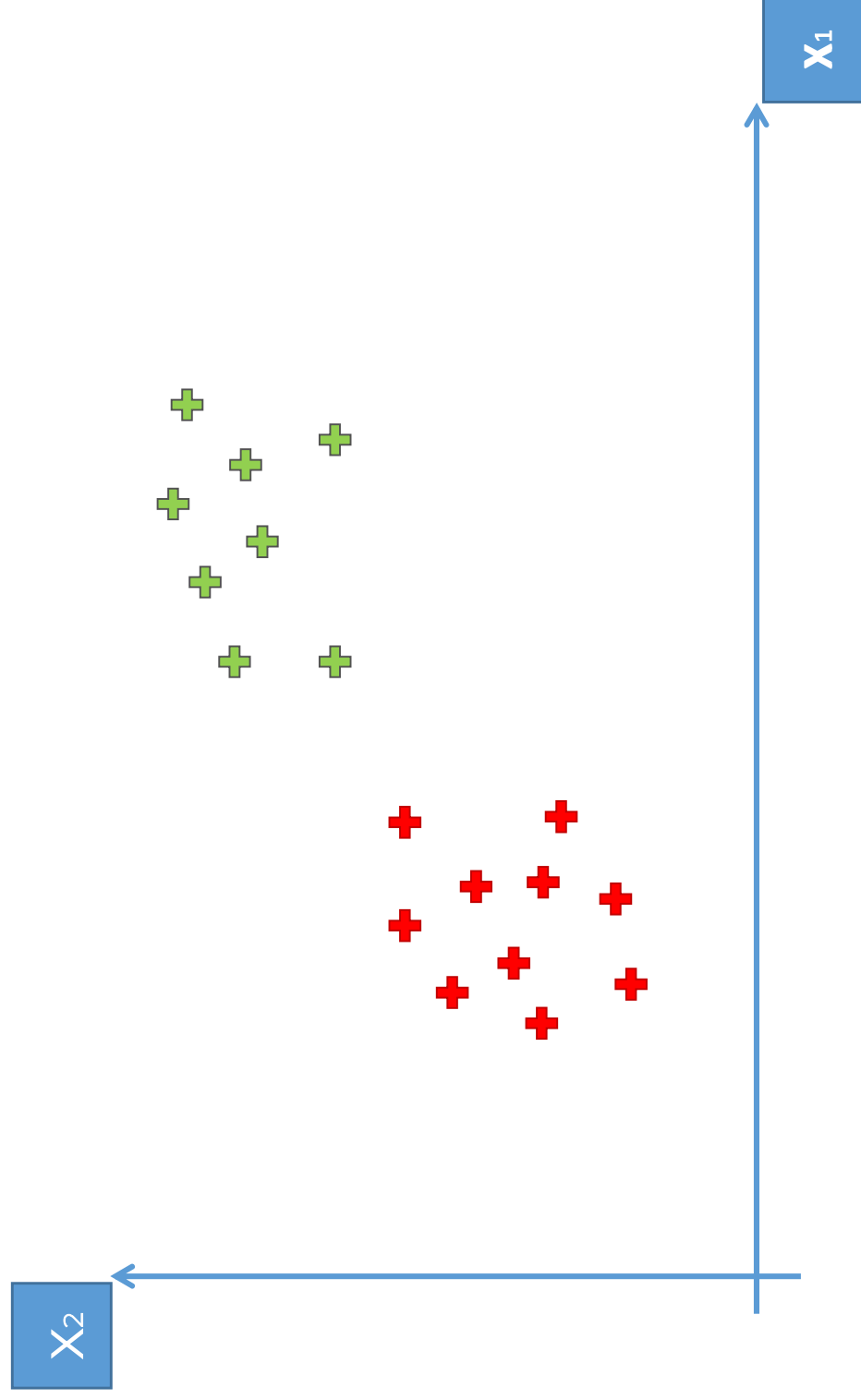
$$\text{Euclidean Distance between } P_1 \text{ and } P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-NN algorithm

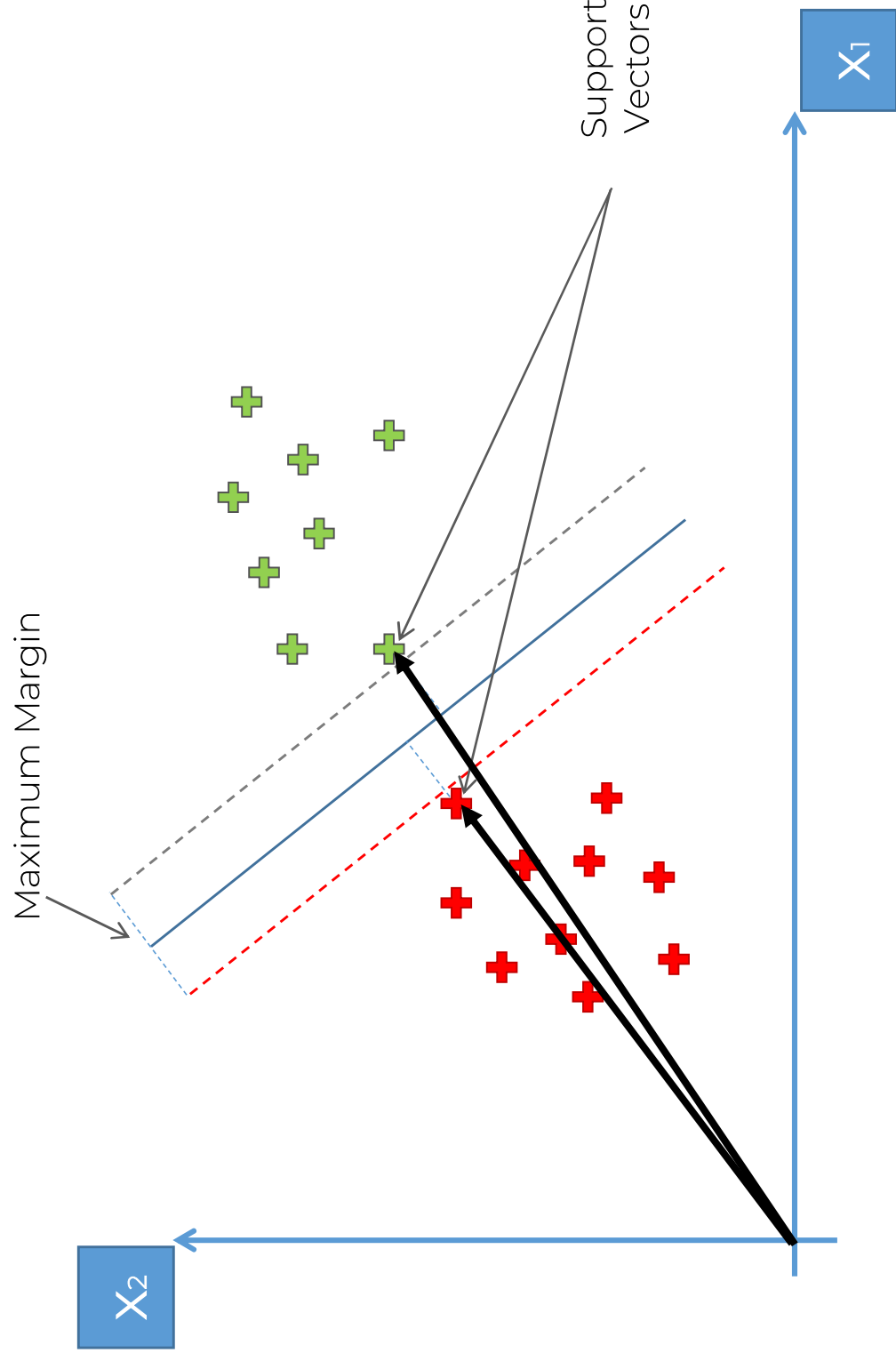


SVM Intuition

How to separate these points?



Support Vectors



Hyperplanes

